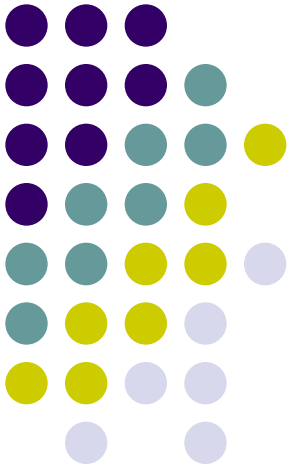
Natural Language Processing

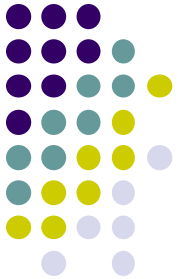
CPSC 488

Transformers:

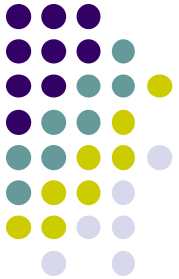
Self-Attention Networks



Lecture Overview



- **Attention mechanisms**
  - Concepts of attention in sequence modeling
  - Cross-attention
  - Self-attention
- **Transformer networks**
  - Neural network architecture
  - Input data preparation
  - Language modeling
  - Sequence modeling



Attention Mechanism

Seq2Seq with the RNN Encoder-Decoder

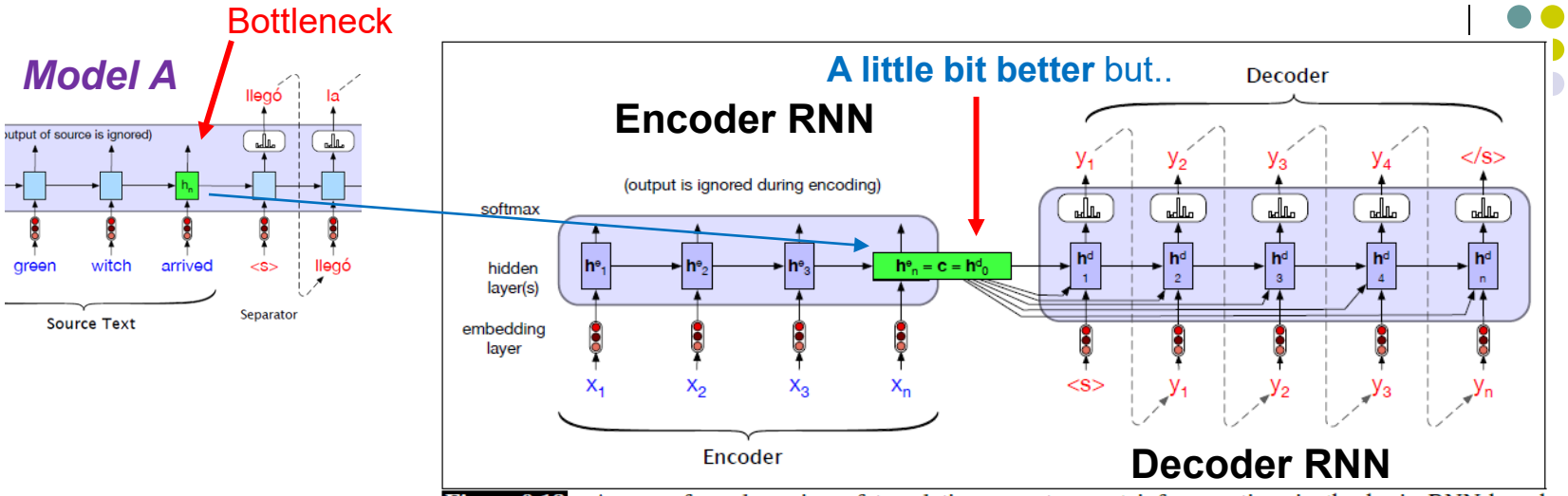


Figure 9.18 A more formal version of translating a sentence at inference time in the basic RNN-based encoder-decoder architecture. The final hidden state of the encoder RNN, h_n^e, serves as the context for the decoder in its role as h_0^d in the decoder RNN.

- Single network layer of encoder-decoder for seq2seq modeling
  - $h_t = g(h_{t-1}, x_t)$ (hidden states at timestep t)
  - $c = h_n^e$ (context vector in encoder that is passed to the decoder)
  - $y_t = f(h_t)$ (decoder autoregressively generates a sequence of outputs.)
- **The problem of the context vector in *model A*:**
  - The hidden state of the last time step of the source text is a bottleneck.
  - The influence of the context vector c will ***wane*** as the output sequence is generated.
  - The information at the early parts of a long sentence, may not be equally represented in the context vector due to vanishing gradients.

The Context Vector at Each Decoding Step

- **Now**, the context vector ***c*** is **available at each decoding.**

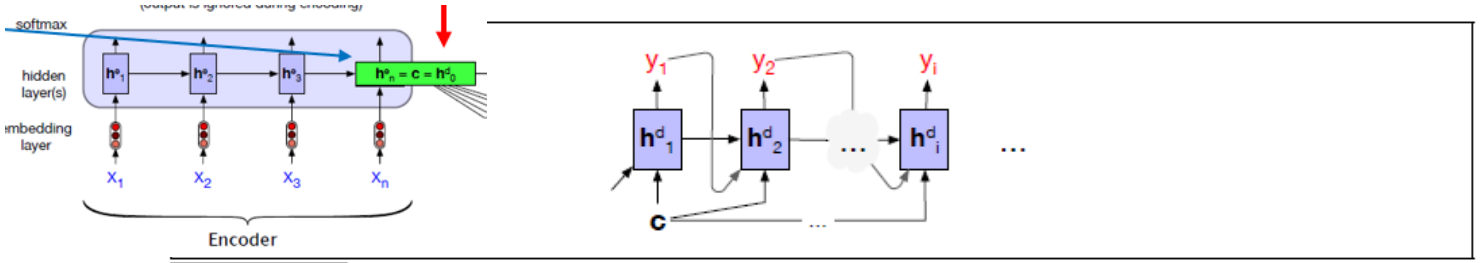


Figure 9.19 Allowing every hidden state of the decoder (not just the first decoder state) to be influenced by the context c produced by the encoder.

- $h_i^d = c$ where $c = h_n^e$ (context from encoder) and $h_t^d = g(\hat{y}_{t-1}, h_{t-1}^d, \boldsymbol{c})$
- $\boldsymbol{y_t} = softmax(z(h_t^d))$ and $\hat{\boldsymbol{y}}_t = argmax_{w \in V} P(w|x, y_1 \ldots y_{t-1})$
  - Compute the most likely output at each time step
- **Problems**:
  - **Encoding** the **entire sequence** into a fixed-length context vector in the final hidden state even if the input sequence is long – bottleneck.
  - The context vector is static for an entire sequence.

Dynamically Derived Context Vector

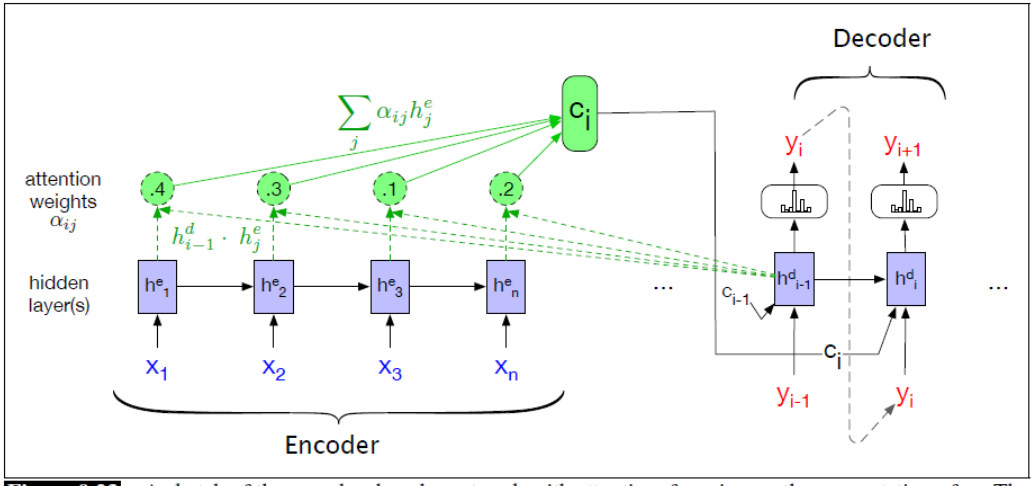


Figure 9.23 A sketch of the encoder-decoder network with attention, focusing on the computation of $\mathbf{c}_i$. The context value $\mathbf{c}_i$ is one of the inputs to the computation of $\mathbf{h}_i^d$. It is computed by taking the weighted sum of all the encoder hidden states, each weighted by their dot product with the prior decoder hidden state $\mathbf{h}_{i-1}^d$.

Seq2Seq modeling:
A sequence is generated *autoregressively* (no teacher forcing)

- **Dynamically derived context vector C_i**
  - The context vector from a weighted sum of all hidden states in the encoder **at time step i**, is different for each token in decoding.
    - The weights focus on ("attend to") a particular part of the entire input sequence, relevant to the **token** the **decoder** is generating.
    - This is the basic idea of "**attention mechanism**".
  - Now the **context vector** is a function of hidden states.
    - The vectors across all hidden states in the encoder gives us the relevance of each encoder state to the current step of the decoder.

Computing the Context Vector

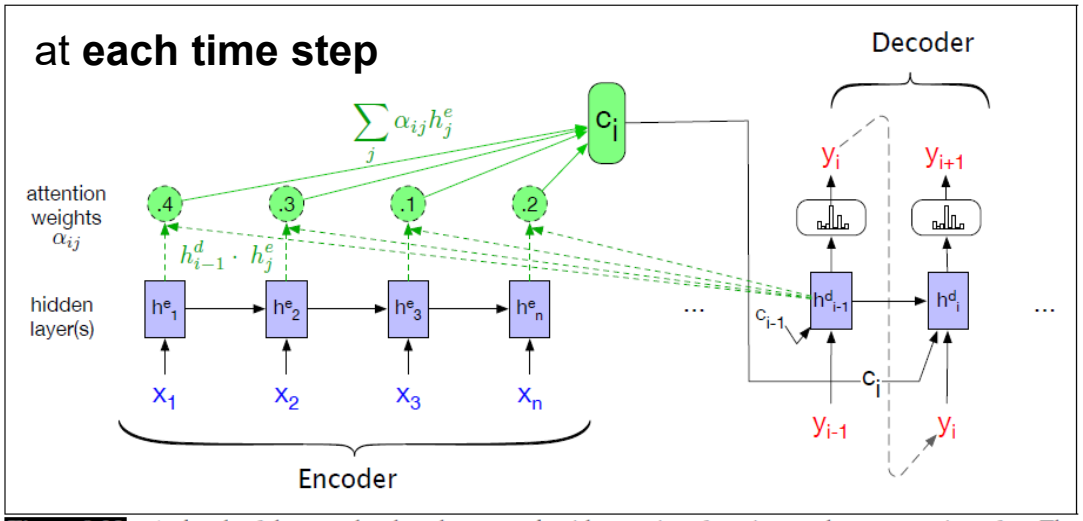


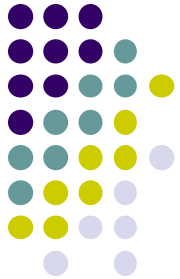
Figure 9.23 A sketch of the encoder-decoder network with attention, focusing on the computation of $\mathbf{c}_i$. The context value $\mathbf{c}_i$ is one of the inputs to the computation of $\mathbf{h}_i^d$. It is computed by taking the weighted sum of all the encoder hidden states, each weighted by their dot product with the prior decoder hidden state $\mathbf{h}_{i-1}^d$.

The **context vector** $\boldsymbol{c_i}$ is created by taking proportion of each hidden vector $\boldsymbol{h_j}$ at each **decoding time step** $\boldsymbol{i}$.

- **Context vector** $\boldsymbol{c_i} = \sum_j a_{ij} h_j^e$ (at time step i and $\boldsymbol{c_i}$ is still a vector)
  - **Attention score**: $\boldsymbol{a_{ij}} = \boldsymbol{softmax}\left(score(h_{i-1}^d, h_j^e)\right) = \frac{\exp(score(h_{i-1}^d, h_j^e))}{\sum_k \exp(score(h_{i-1}^d, h_k^e))}$
    - The similarity between two vectors from encoder and decoder:
      $\boldsymbol{score}(h_{i-1}^d, h_j^e) = h_{i-1}^d \cdot h_j^e$ (**dot product**)
    - The normalized weights indicate proportional relevance of each encoder hidden state, h_j^e to the prior hidden decoder state, h_{i-1}^d.
    - This is basic idea of "attention mechanism" (D. Bahdanau et al. 2014)

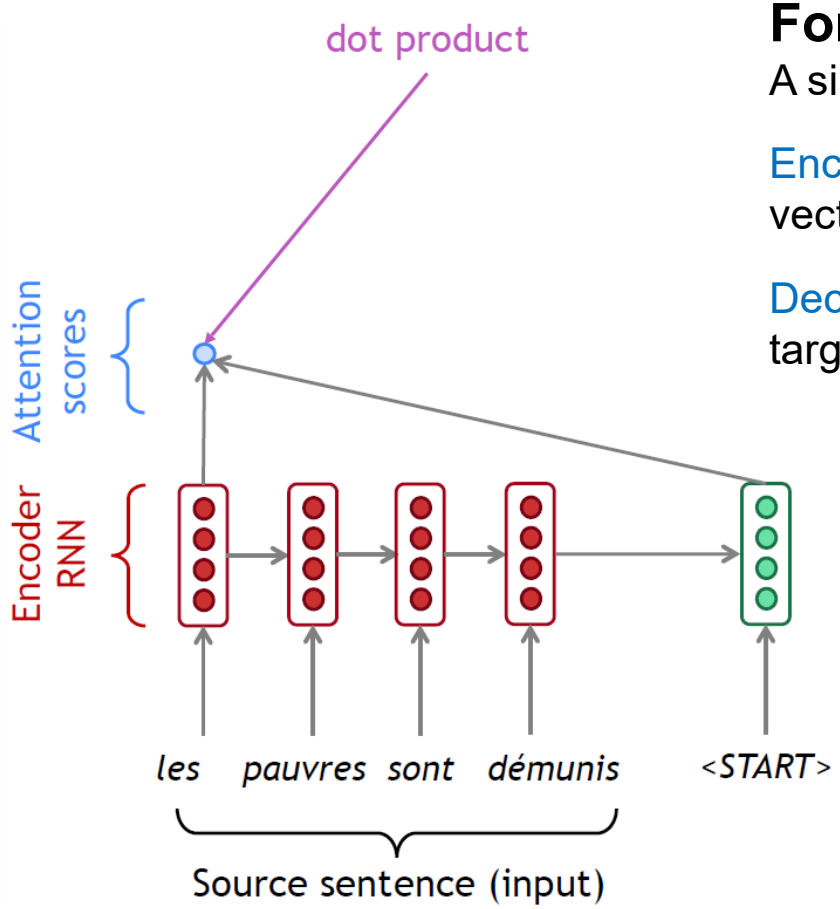
Attention in Seq2Seq Modeling

(D. Bahdanau, K. Cho et al., **2014**)



- **What is the basic idea of attention?**
  - The model focuses on the **most relevant parts** of the **input sequence** instead of treating all input words equally.
    - Unlike traditional RNN, at each time step of decoding, the decoder focuses on a particular part of source input sequence.
    - This depends on the decoder's current hidden state (notion of what you are trying to decode).
    - Usually implemented as a probability distribution over the hidden states of the encoder.
- **Key benefit**
  - **Resolving** the issues of encoding the entire sequence into a static and fixed-length context vector
    - The **context vector is dynamically derived** at **each decoding time step**.

Example: Seq2Seq with *Attention* (1)



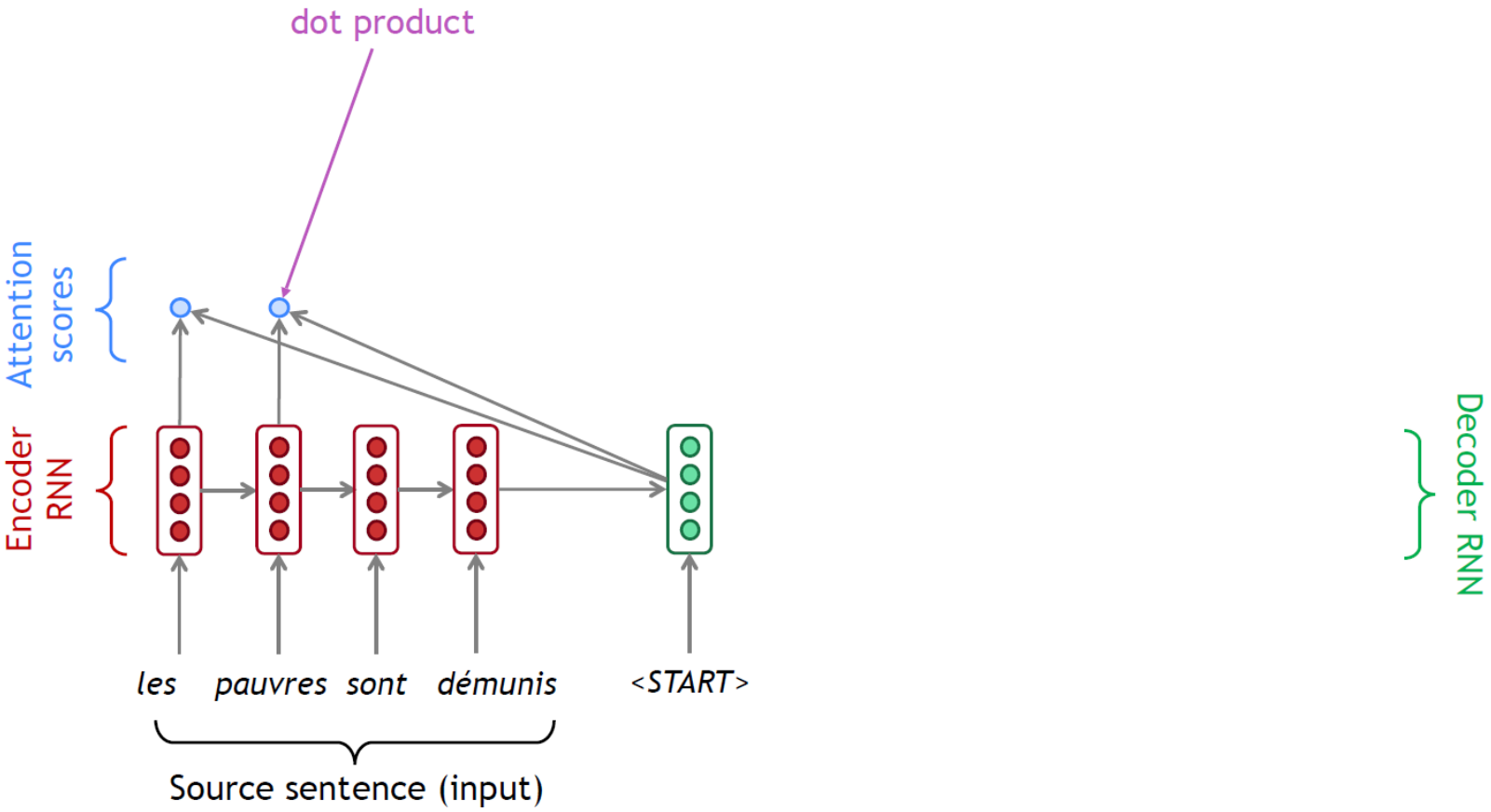
For Neural Machine Translation

A single neural network architecture

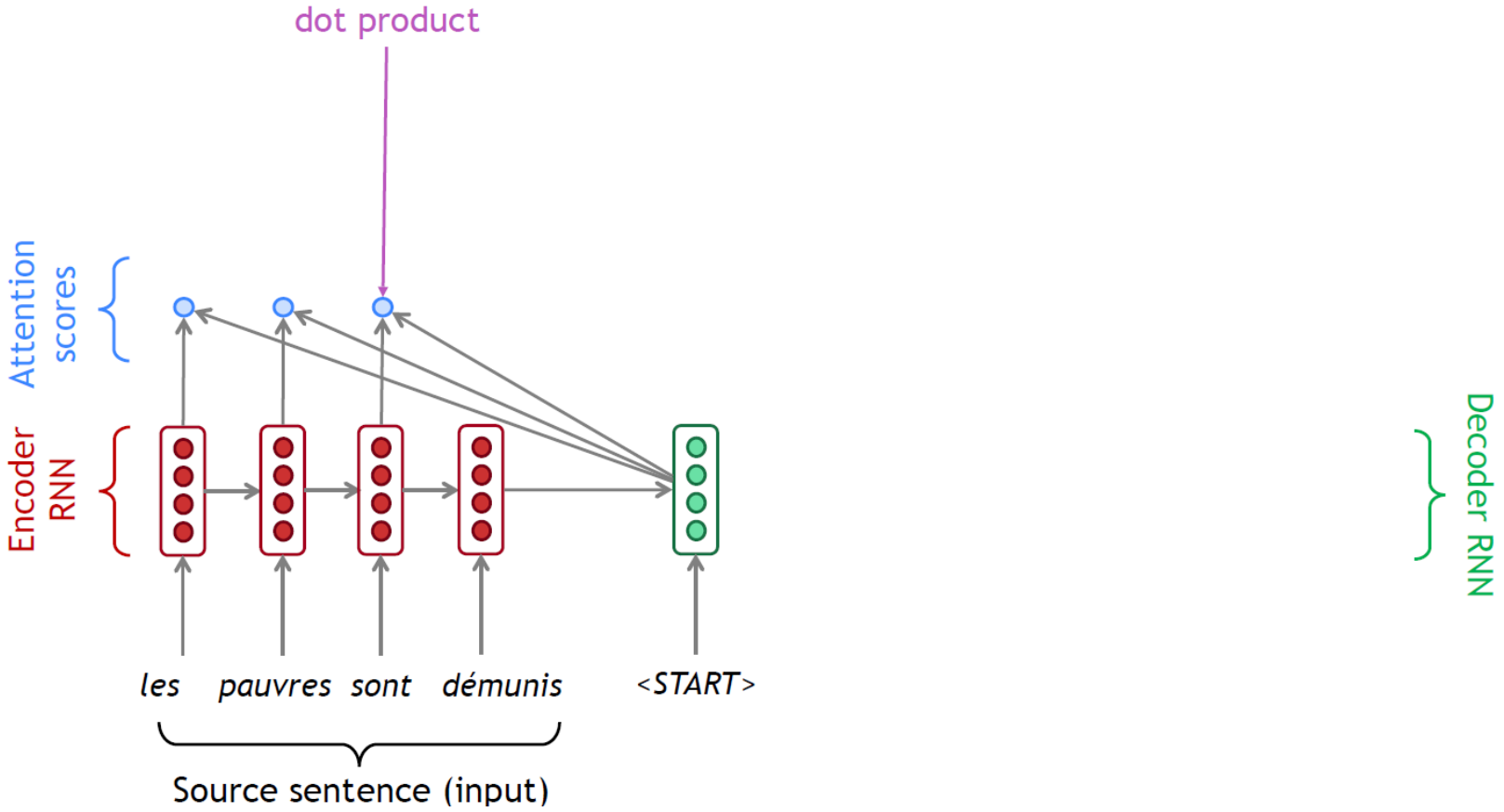
Encoder: Convert source input sentence into a vector/matrix

Decoder: Convert encoding into a sentence in a target language (output)

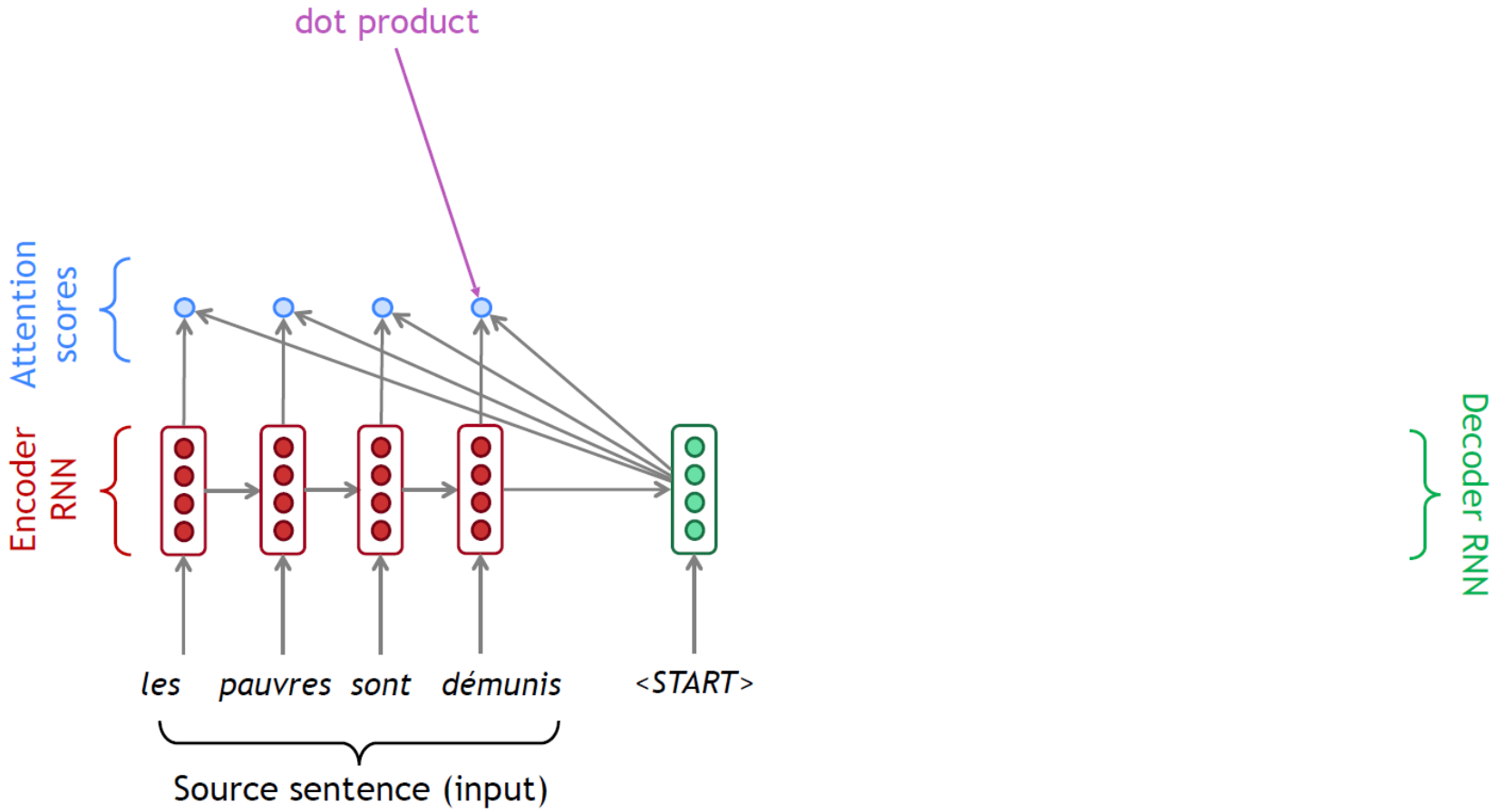
Example: Seq2Seq with *Attention* (2)



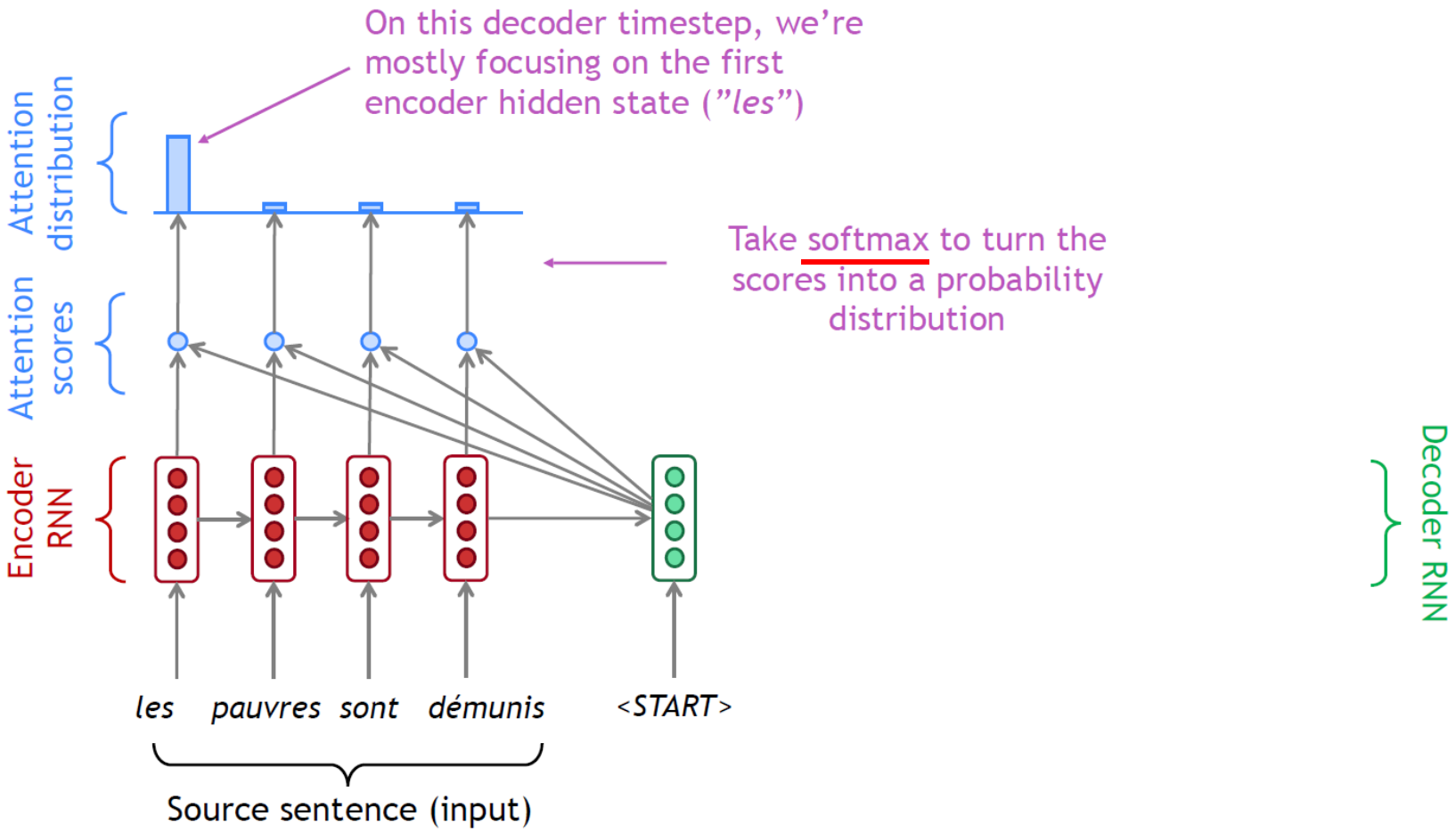
Example: Seq2Seq with *Attention* (3)



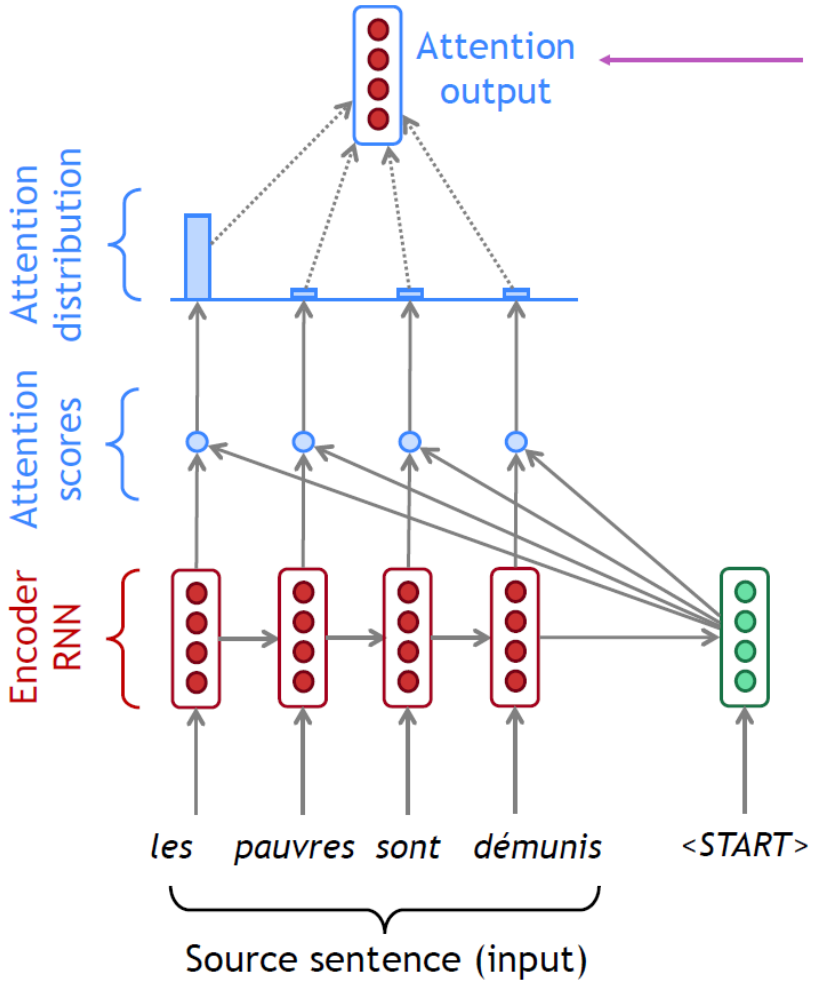
Example: Seq2Seq with *Attention* (4)



Example: Seq2Seq with *Attention* (5)



Example: Seq2Seq with *Attention* (6)

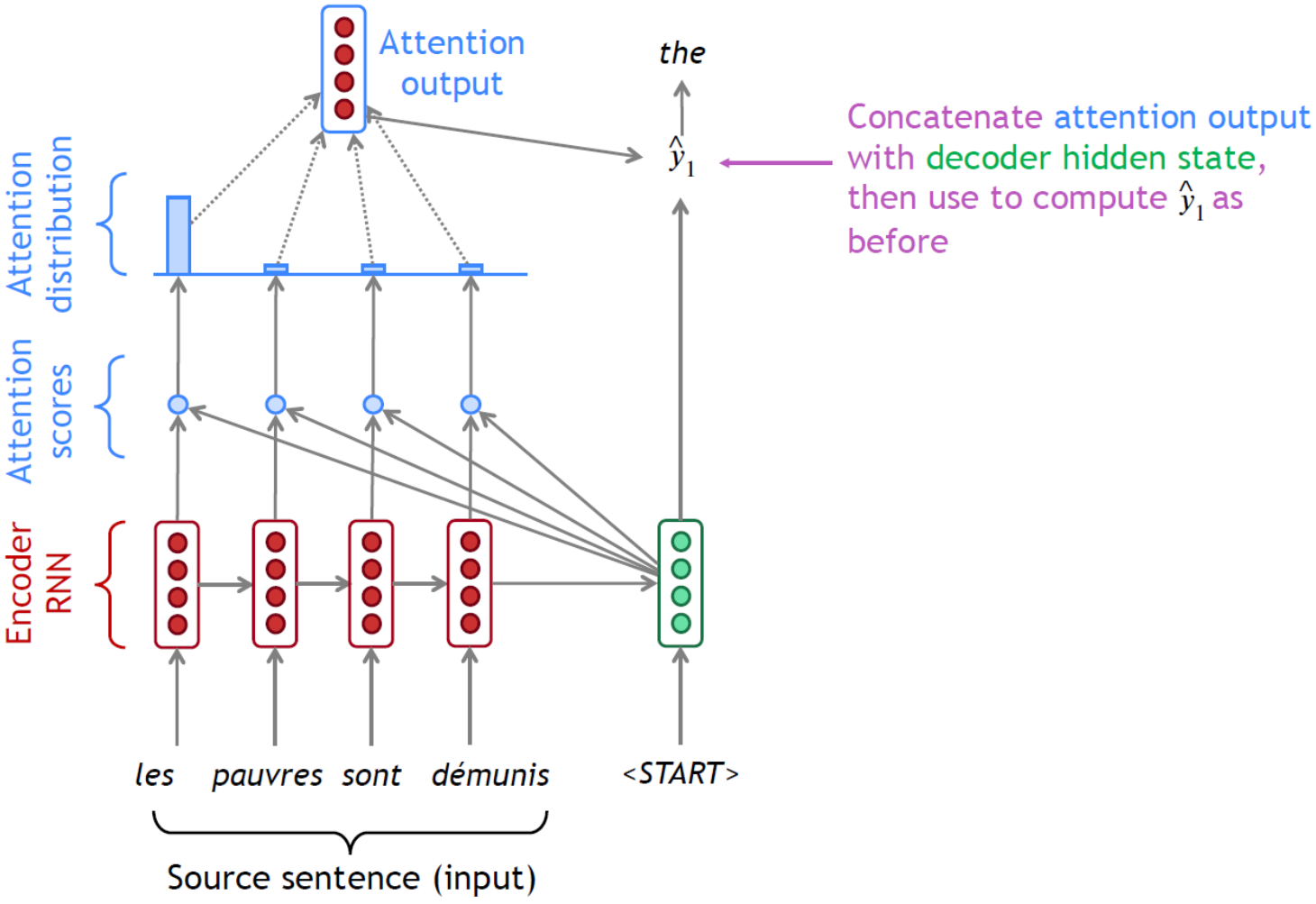


Use the attention distribution to take a **weighted sum** of the encoder hidden states.

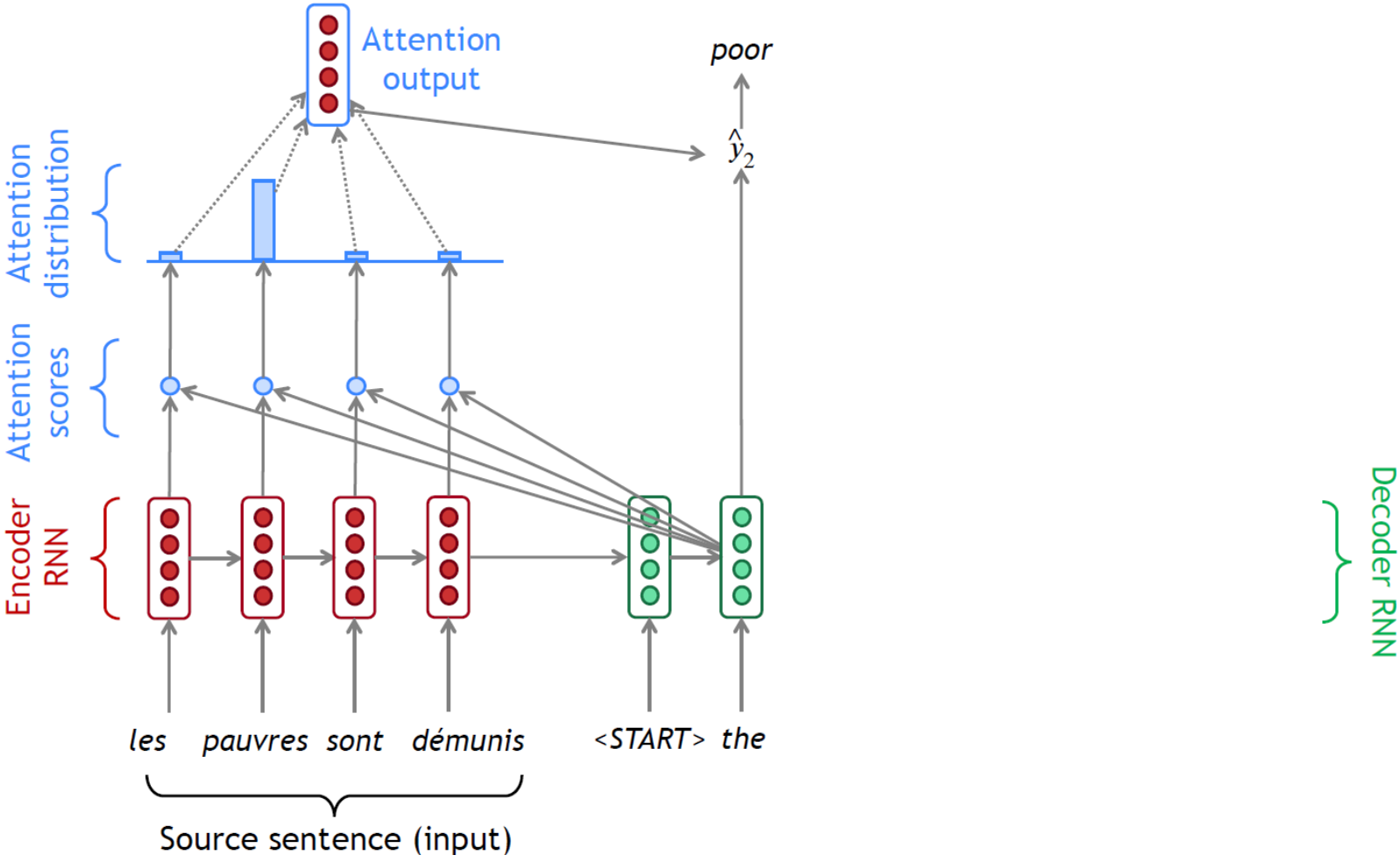
The attention output mostly contains information the hidden states that received high attention.

Decoder RNN

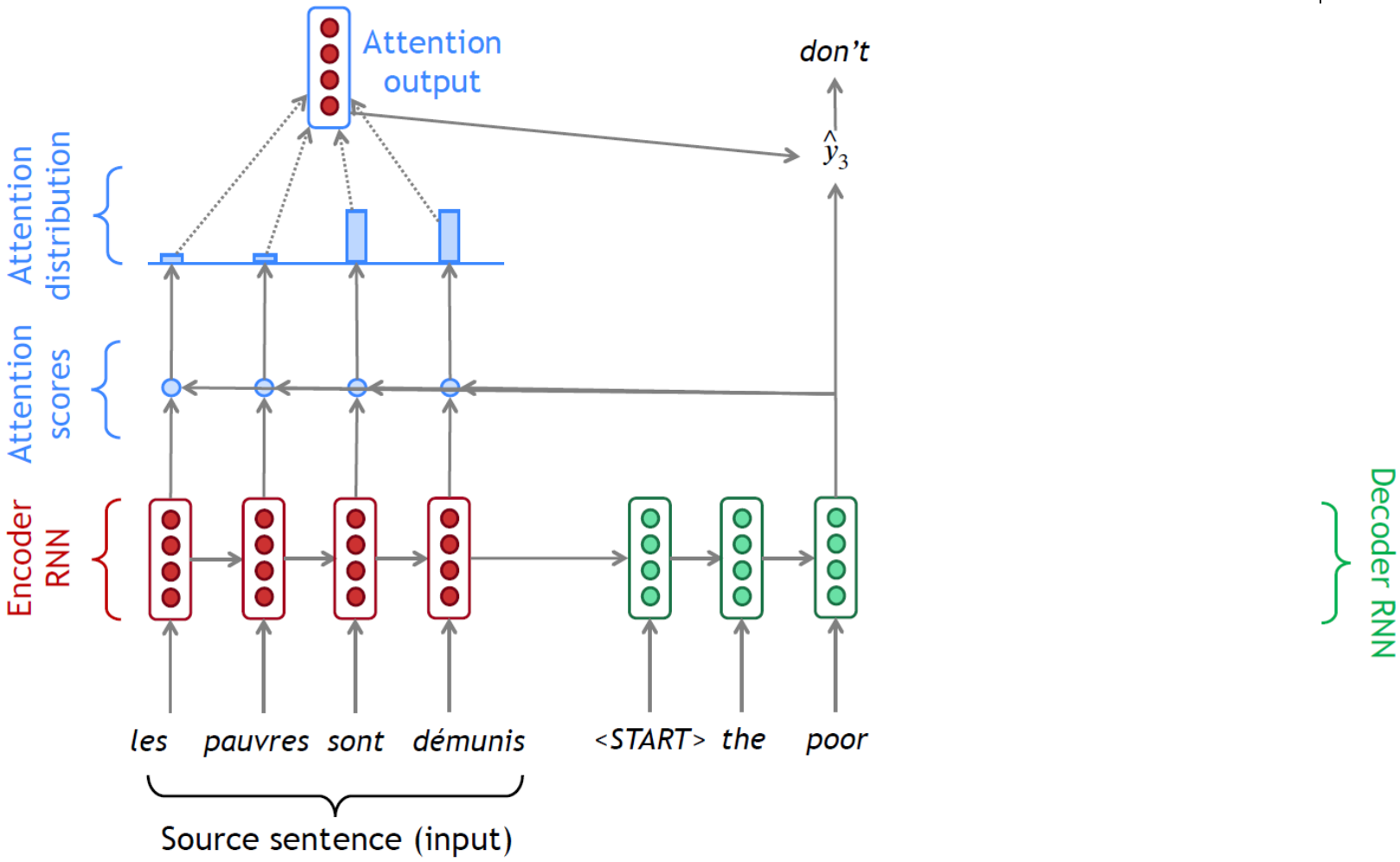
Example: Seq2Seq with *Attention* (7)



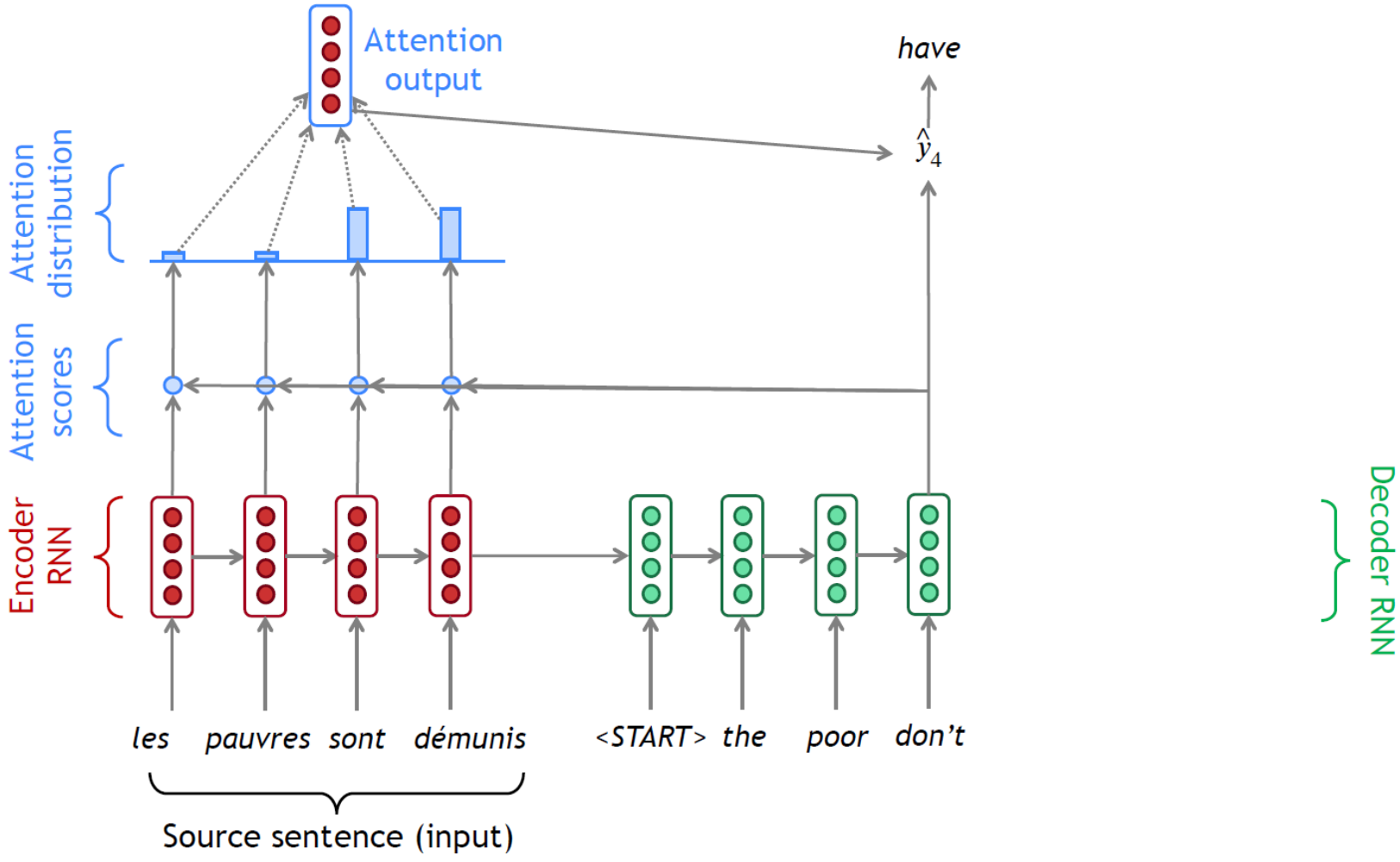
Example: Seq2Seq with *Attention* (8)



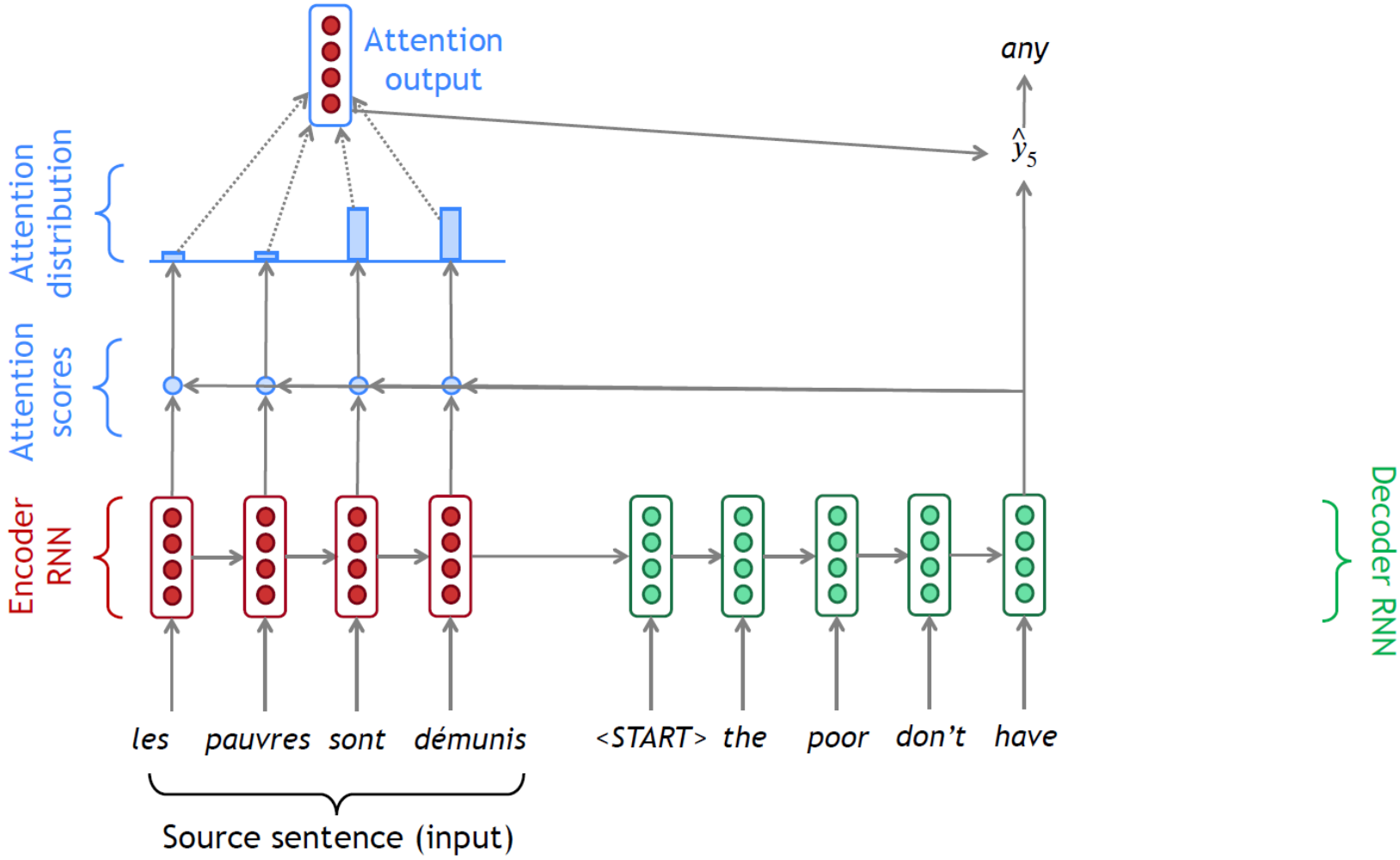
Example: Seq2Seq with *Attention* (9)



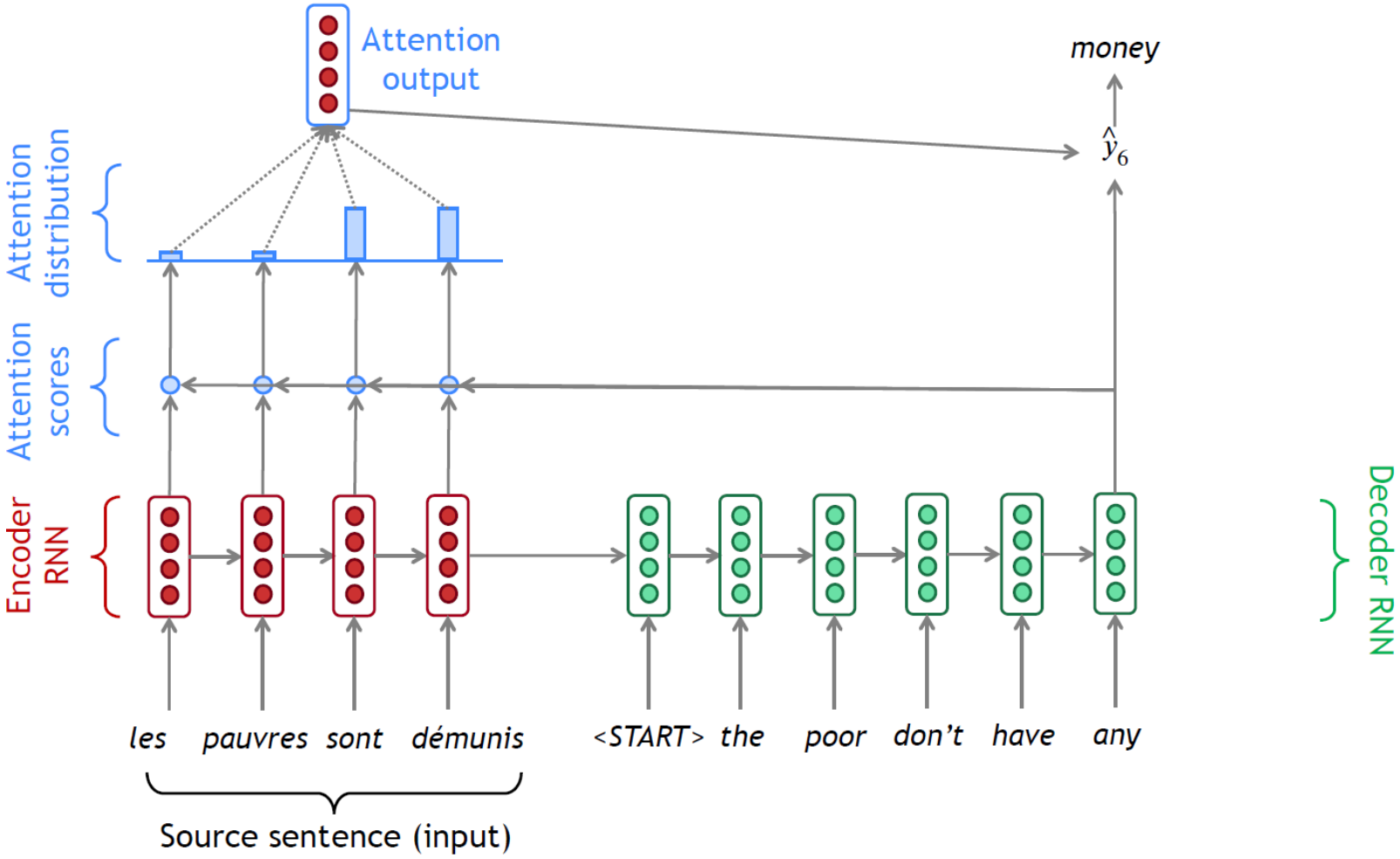
Example: Seq2Seq with *Attention* (10)



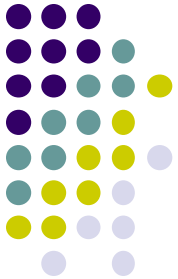
Example: Seq2Seq with *Attention* (11)



Example: Seq2Seq with *Attention* (12)

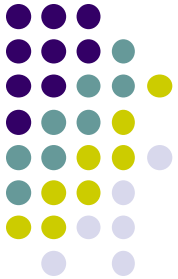


Various Ways to Compute Attention

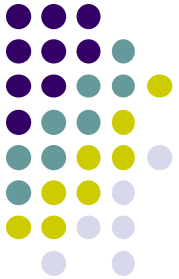


- For **encoder** hidden states $h_1, h_2, \ldots, h_n$ and **decoder** hidden state h_z, the attention score can be computed:
  - **Additive attention (**concatenate vectors**,** Bahdansu**)**
    - $e_i = g(h_i, h_z) = v^T \tanh(W_1 h_i + W_2 h_z) \in \mathbb{R}$ where W_i are weight matrices and v is a weight vector (concatenate two vectors)
  - **Dot product attention**
    - $e_i = g(h_i, h_z) = {h_z}^T h_i \in \mathbb{R}$ (dot product in Transformer, no concatenation)
  - **Multiplicative attention**
    - $e_i = g(h_i, h_z) = {h_z}^T W h_i \in \mathbb{R}$ where W is a weight matrix via training (computing alignment by dot product and concatenate two vectors (Luong)

Summary of Attention Mechanism



- **Attention mechanism by Bahdanau et al.**
  - At each decoding step, compute a score between all encoder hidden states and the current decoder hidden state.
  - Compute single context vector
  - Developed for Seq3seq model, mostly for machine translation with RNNs
- **Limitations**
  - Sequential computational bottleneck
    - Both encoder and decoder relying on RNNs (inherently sequential and limiting parallelization)
  - Inefficient for large sequences
    - Computing attention at every decoder step across all encoder states.
  - Long-range dependency issues
    - Even with attention, RNNs struggle with long sequences
  - Single context vector limitation
    - Single context vector per decoding step may not capture multi-faceted dependencies



Self-Attention Mechanism

Self-Attention

(Vaswani et al., "**Attention is All You Need**", Google Brain, 2017)

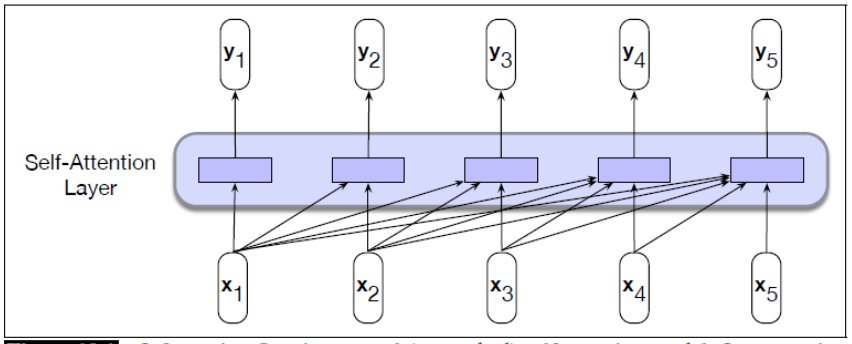
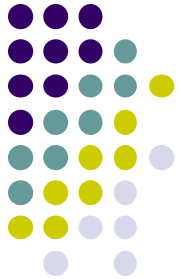


Figure 10.1 Information flow in a causal (or masked) self-attention model. In processing each element of the sequence, the model attends to all the inputs up to, and including, the current one. Unlike RNNs, the computations at each time step are independent of all the other steps and therefore can be performed in parallel.

- **The impact of this paper**
  - Introduced a "novel" architecture "**Transformer**" for seq2seq modeling
  - A significant shift in how neural networks are applied to sequence modeling, particularly in NLP and beyond.

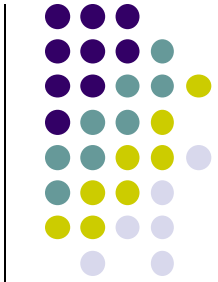
- **Bahdanau et al.'s attention**
  - Focuses on the relationship between *separate* input sequences (from the encoder) and output sequences (from the decoder).
- **Self-attention**
  - An input is an entire sequence. **Each position** attends to **all other positions** within the same sequence.
  - It still attends to the **most** relevant parts in a sequence.
    - Attention score function (dot product) assigns higher weight to the most relevant tokens
  - Still limited to the window size (**but** no sliding window or recurrent connection)
    - Used **various ideas** such as token embedding, positional embedding, matrix projection (encoding), sparse attention, etc.

Cross-Attention vs. Self- Attention



- **Concept of attention** (Bahdanau et al., 2014)
  - A method to determine how much input contributes to output
  - First introduced in Bahdanau's attention mechanism
- **Cross-attention** (Bahdanau's attention)
  - Focus on the most relevant parts of **another sequence**
  - Attention scores determine how each input token contributes to the current output token
    - E.g., in seq2seq modeling, a decoder predicting the next word attends to all encoder outputs
- **Self-attention**
  - A special case of attention (core mechanism used in **Transformer**)
  - Each token computes attention over all tokens in the same sequence
    - Allowing it to capture long-range dependencies

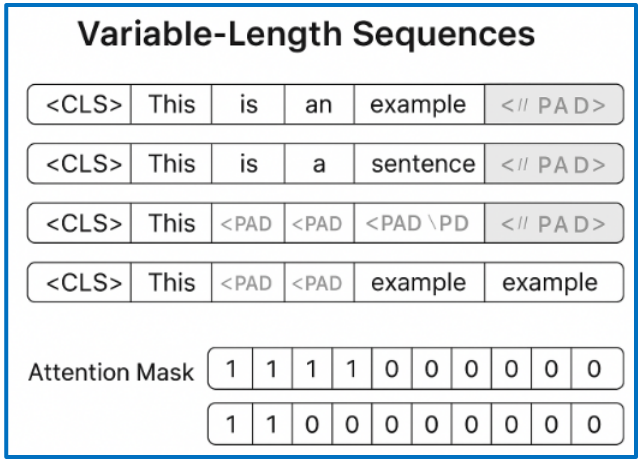
Why Self-attention?



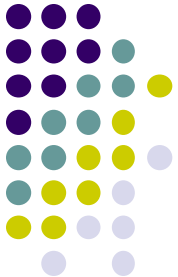
- **The main problems addressed using self-attention**
  - *No recurrent connections* – no RNNs
    - resolving the inherent problems of sequential processing in RNN
  - For *high parallelism* and *computational efficiency*
    - The entire input sequence is fed into the network at once, not one word at a time.
    - Each pair can be independently computed – easier parallelism.
  - *Capturing long-range or global dependencies*
    - By allowing all positions in the sequence to interact directly
  - Flexible contextual representation with *multiple contexts*
    - Pairing with the other words enables capturing multiple perspectives in a sequence
  - *Memory* and *context* are *dynamically adjusted* for each input
    - Context window size - only limited by computational resources
- **The two questions still need to be answered:**
  - How does it handle variable-length input (still limited window size)?
  - How does it handle the order of words without recurrent connections?

Handling Variable-length Input

- **Fixed long enough window size**
  - The size should be long enough to hold long input sequences.
    - Window size is a hyperparameter.
  - Long and short input sequences
    - Short sequences are padded with a special **<PAD>** token so that all sequences in the batch have the same length.
    - Padding tokens are **masked** (**attention mask**). So, the model ignores them when computing attention score.
- **Batching input sequences and feeding one batch at a time**
  - For efficiency, sequences are grouped into batched of roughly similar lengths to minimize the amount of padding.
  - Each batch sees the full sequence at once.

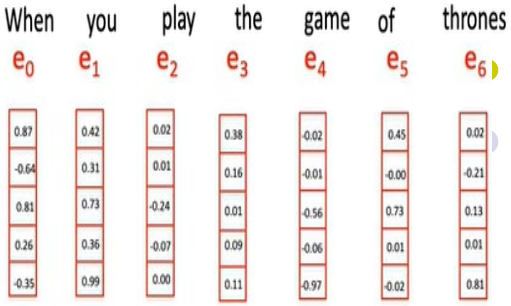


Input Batch



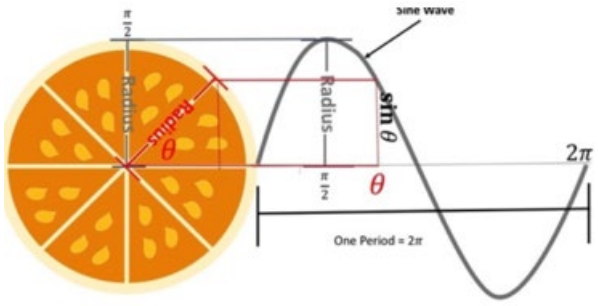
- **How is a batch created?**
  - For efficiency, sequences can be sorted or bucketed by length before batching, to minimize the amount of padding and increase parallelism
    - e.g., CUDA's hierarchical parallelisms (e.g., batch-, sequence-, matrix-, vector-level)
  - The max. sequence length can be different from batch to batch.
    - The total number of tokens for each batch in the original Transformer is **~25k**.
- **Input tensor shape**: batch size x sequence length x dimension
  - The sequences are stacked into a tensor and the entire batch tensor plus mask is fed all at once
    - E.g., a batch of 2 sequences with embedding size 512 and sequence length 5, the tensor shape is (2, 5, 512)
  - But contextual relationships (self-attention scores) are computed within each sequence (not within the batch).
  - The network parameters (weight matrices) are shared across all batches and updated globally.

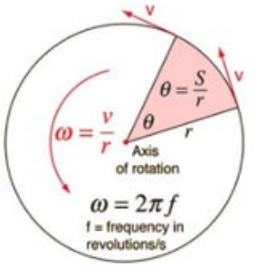
Maintaining the Word Order



- **How to maintain the order of words**
  - Unlike RNN, Transformer has no position for each token in a sequence.
  - But the order of a word in a sentence matters.
    - "Even though she did not win the award, she was satisfied".
    - "Even though she did win the award, she was not satisfied".
  - How do we add the position information to the input embeddings?
    - **A simple idea**: e (word vector) + p (scalar position) will not work!
  - Better idea
    - Input token vector: (embedded token vector) + (position vector)
- **Positional Embeddings (PE)**
  - An encoded presentation of token's positions in a sequence, assigning each token a unique vector so that the model can distinguish word order
  - **Desirable properties of token position**
    - Deterministic and unique position within a range regardless of the input length
    - Possible estimation of the distance between two token's positions
    - Varying non-linearity but not growing too big by the input length
    - Learnable representation and enabling parallel processing

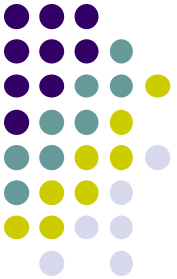
Wave Functions



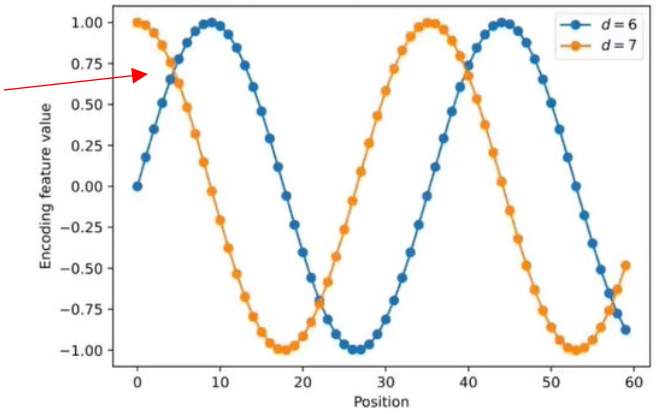


- **Sine and Cosine functions**: $y = \sin(\theta)$, $y = \cos(\theta)$
  - Functions oscillate between -1 and +1
  - Sine starts at 0 and Cosine starts at 1 ($90°$ phase shifted)
  - Angle θ is commonly measured in radians. So, full circle $360° = 2\pi$ radians.
- **General wave function** $y(t) = \sin(\omega t + \phi)$ where ϕ = phase shift
  - As time t passes, the angle changes $\omega = \frac{\Delta\theta}{\Delta t}$ => $\theta(t) = \omega t$
  - Angular frequency ω = radians per second
  - Frequency f = number of cycles per second (Hz): $f = \frac{\omega}{2\pi}$, so $\omega = 2\pi f$
- **Angular frequency and wavelength**
  - The frequency of a wave is inversely proportional to its wavelength.
    - Low ω → long wavelength, slow oscillation, High ω → short wavelength, fast oscillation
- **Period** $T = \frac{1}{f} = \frac{2\pi}{\omega}$ (time taken to compete one full cycle)
  - The frequency of a wave is inversely proportional to its period.
    - Short period → wave changes rapidly, long period → wave changes slowly.
    - Low ω → low f → long wavelength, High ω → high f and short wavelength

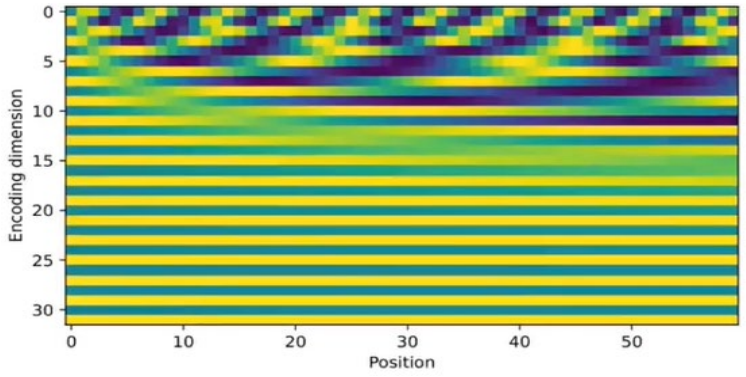
Positional Embeddings (PE) using the Wave Function (Vaswani et al., 2017)



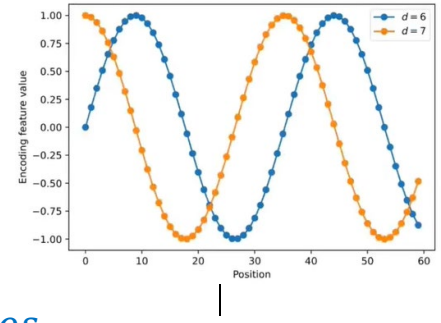
- $PE(pos, 2i) = sin(\frac{pos}{n^{2i/d}})$ and $PE(pos, 2i+1) = cos(\frac{pos}{n^{2i/d}})$
  - pos is the **token position**, $0 \leq pos < L/2$ where L is the sequence length.
  - i is the index of the dimension for a specific position, $0 \leq i < d/2$.
  - d is the dimension of embedding matrix.
  - n is a scaling factor (10,000 used in the original transformer).
- The different **position** corresponds to different **waves** that encode each position into a **unique vector**, satisfying the desirable properties of position.



If we stack all these into $PE \in \mathbb{R}^{pos \times d}$

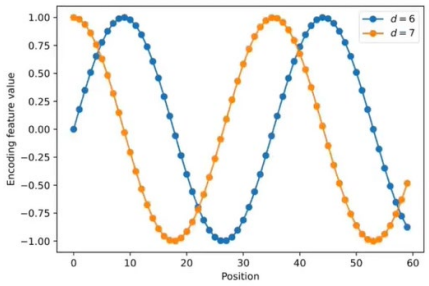


Example: Positional Embedding



- $\boldsymbol{PE(pos, 2i) = sin(\frac{pos}{n^{2i/d}})}$ and $\boldsymbol{PE(pos, 2i+1) = cos(\frac{pos}{n^{2i/d}})}$
  - From $y(t) = \sin(\omega t + \phi)$, $\boldsymbol{w_i = \frac{1}{n^{2i/d}}}$ and $\boldsymbol{t = pos}$
  - i = embedding dimension index, $0 \leq i < d/2$
  - ω_i = angular frequency of the wave in that dimension
  - $n = 1000$ in original Transformer is the range of wavelengths
  - $\boldsymbol{PE_{pos,2i} = sin(\omega_i \cdot pos)}$ and $\boldsymbol{PE_{pos,2i+1} = cos(\omega_i \cdot pos)}$
    - The different frequencies encode relative positions.
    - Low dimensions: high frequency (inverse of ω_i), sensitive to short-range position differences
    - High dimensions: low frequency, sensitive to long-range position differences
- **Example**:
  - Let $d = 4$, then two wave lengths to exponents $2i/d$ for $i = 0,1$:
  - $i = 0$: $1000^{2\cdot 0/4} = 0 \rightarrow 1000^0 = 1 \rightarrow \sin\left(\frac{1}{1} \cdot pos\right) = \sin(pos)$
  - $i = 1$: $1000^{2\cdot 1/4} = 0.5 \rightarrow 1000^{0.5} = 100 \rightarrow \sin\left(\frac{1}{100} \cdot pos\right) = \sin(0.01pos)$
  - For $pos = 1$ with $d = 4$ vector: $[\sin(1), \cos(1), \sin(0.01), \cos(0.01)]$

Example: Positional Embeddings



Sequence	Index of token	Positional Encoding Matrix			
When	0	P_{00}	P_{01}	…	P_{0d}
you	1	P_{10}	P_{11}	…	P_{1d}
play	2	P_{20}	P_{21}	…	P_{2d}
the	3	P_{30}	P_{31}	…	P_{3d}

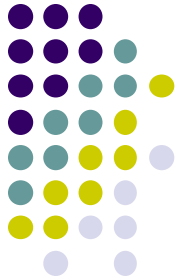
Sequence	Index of token, k	Positional Encoding Matrix with d=4, n=100			
		i=0	i=0	i=1	i=1
When	0	P_{00}=sin(0) = 0	P_{01}=cos(0) = 1	P_{02}=sin(0) = 0	P_{03}=cos(0) = 1
you	1	P_{10}=sin(1/1) = 0.84	P_{11}=cos(1/1) = 0.54	P_{12}=sin(1/10) = 0.10	P_{13}=cos(1/10) = 1.0
play	2	P_{20}=sin(2/1) = 0.91	P_{21}=cos(2/1) = -0.42	P_{22}=sin(2/10) = 0.20	P_{23}=cos(2/10) = 0.98
the	3	P_{30}=sin(3/1) = 0.14	P_{31}=cos(3/1) = -0.99	P_{32}=sin(3/10) = 0.30	P_{33}=cos(3/10) = 0.96

- **Why Sine and Cosine together?**
  - Cosine = Sine with $90°$ phase shift to ensure unique encoding
  - Can compute **relative positions** linearly: $\sin(a+b) = \sin(a)\cos(b) + \cos(a)\sin(b)$
  - Attention layers can **infer distance** between positions using this property.
  - Similar to the binary representation of number

$$\begin{bmatrix} \sin(w_1 \cdot pos) \\ \cos(w_1 \cdot pos) \\ \sin(w_2 \cdot pos) \\ \cos(w_2 \cdot pos) \\ \cdots \\ \sin(w_{d/2} \cdot pos) \\ \cos(w_{d/2} \cdot pos) \end{bmatrix}_d$$

from $\sin(w_i \cdot pos)$
where $w_i = \frac{1}{n^{2i/d}}$

Other Ideas for Positional Embeddings



- **Pros and cons of PE using the wave function**
  - Pros
    - Ensure smooth positional transitions due to the continuous nature of sine and cosine functions without additional parameters to learn
  - Cons
    - Fixed and non-adaptive to the specific data or task
    - Might not capture complex positional relationships as effective as learned embeddings
- **Other positional embedding ideas proposed later**
  - Learnable positional embeddings
    - Treat positions similarly to tokens, learning a unique embedding vector for each position during training
  - Relative positional embeddings
    - Encode the relative distances between tokens rather than absolute positions.
    - Useful in capturing local dependencies and patterns
  - Rotary positional embeddings (RoPE)

Transformer: Self-Attention Network



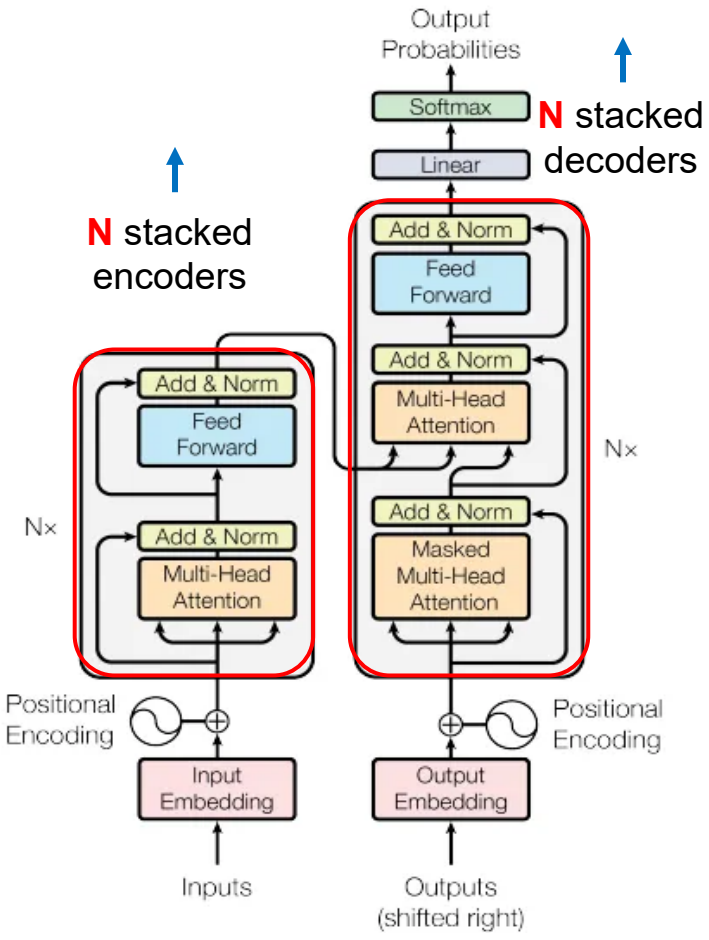
Vs



The Transformer Architecture

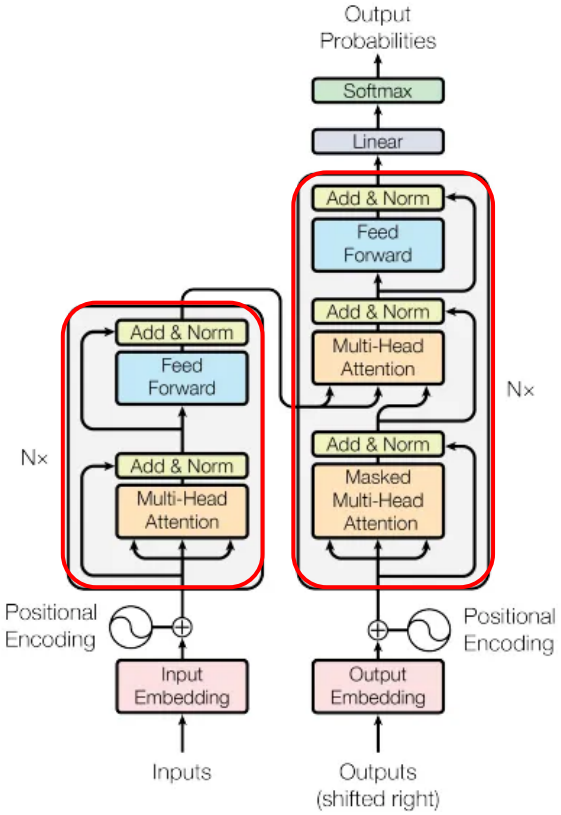
(Vaswani et al., "Attention is All You Need", Google Brain, 2017)

For ***Seq2Seq* modeling**



- **Encoders**
  - **To encode** the *entire sequence* into a continuous representation of fixed-length context vector.
    - **Self-attention layer** captures the semantic and context relationships in the input sequence.
    - **Feedforward network layer** transforms the data from the self-attention layer to help extract and encode information via non-linear transformation.
    - **Normalization** is to stabilize the learning process.
- **Decoders**
  - **To generate** the output sequence by taking the encoded representation, one token at a time in an autoregressive manner.
    - **Self-attention layer** captures the semantic and context relationships in **both** input and output sequences.
    - **Feedforward network layer** transforms the data further based on the encoded input and previously generated output.
    - **Normalization** is to stabilize the learning process.

Input Embedding
+
Positional Embedding

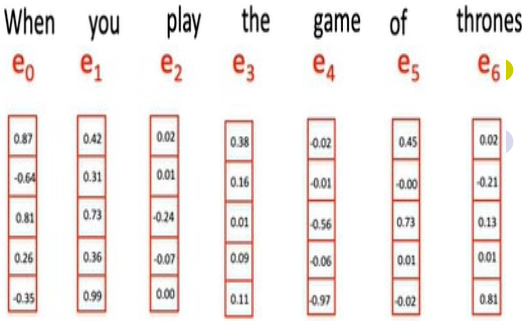




Will the **resulting vector** maintain the positional information?

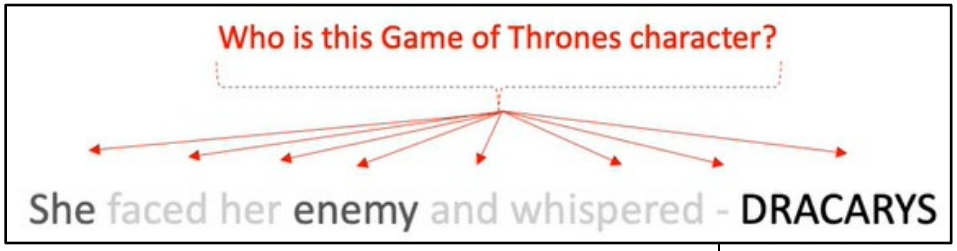
- **Input layer**: self-attention layer
  - The input embedding added with positional embedding will be fed to the multi-headed self-attention layer.

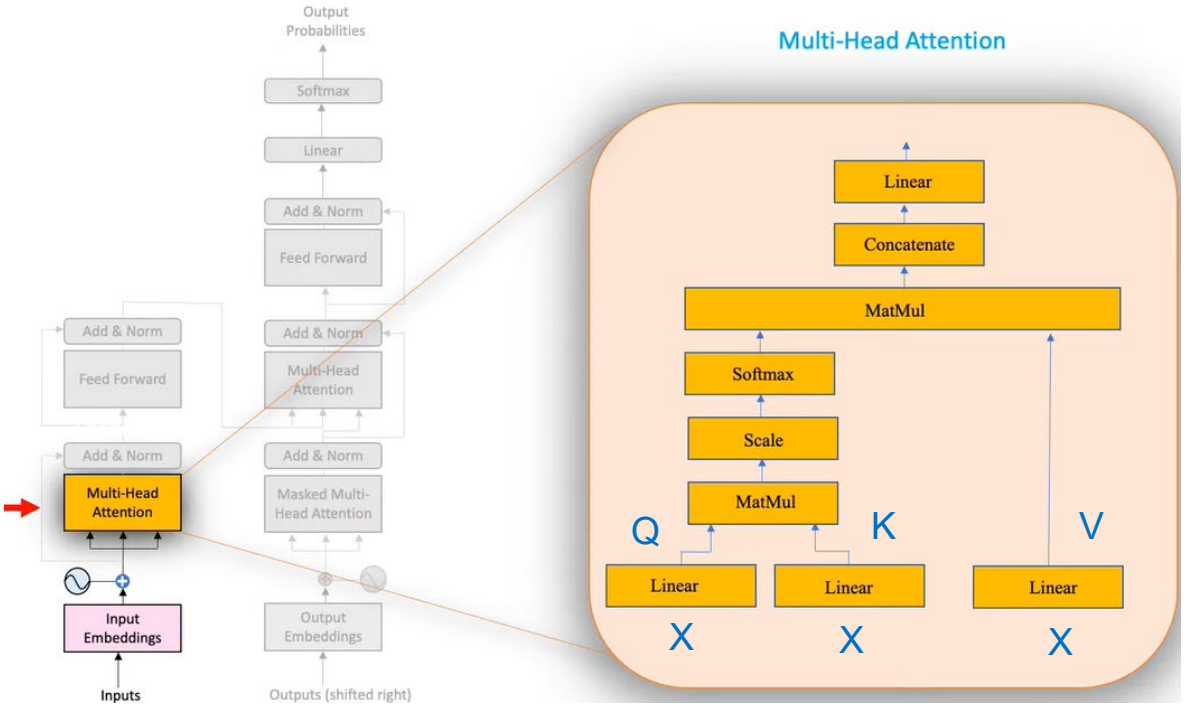
Input Word Embeddings



- ***Static word* embeddings**
  - Static word embedding $E \in \mathbb{R}^{V \times d}$ from vocabulary-wide training
    - V = vocabulary size
    - d = embedding dimension
    - Word embeddings are created similarly to Word2vec based on the model's (128 – 1024 dimensions of) linguistic features (frequency, semantic similarity, context, etc.)
    - and used as a lookup table for one vector per token
- ***Dynamic contextual* embeddings** (used in Transformer)
  - Static word embeddings will be combined with positional embeddings.
  - The combined embeddings get **updated** via full-model training, creating dynamic contextual embeddings
    - The self-attention layer transforms them based on context words.
    - Output embeddings will be context-specific, e.g., vectors for "bank" will be different in different contexts, money-bank and river-bank.

Self-attention Layer



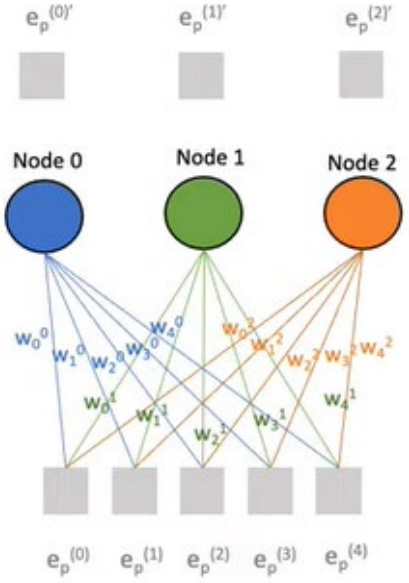


Linear layer is a Fully connected Feedforward Network (FFN).

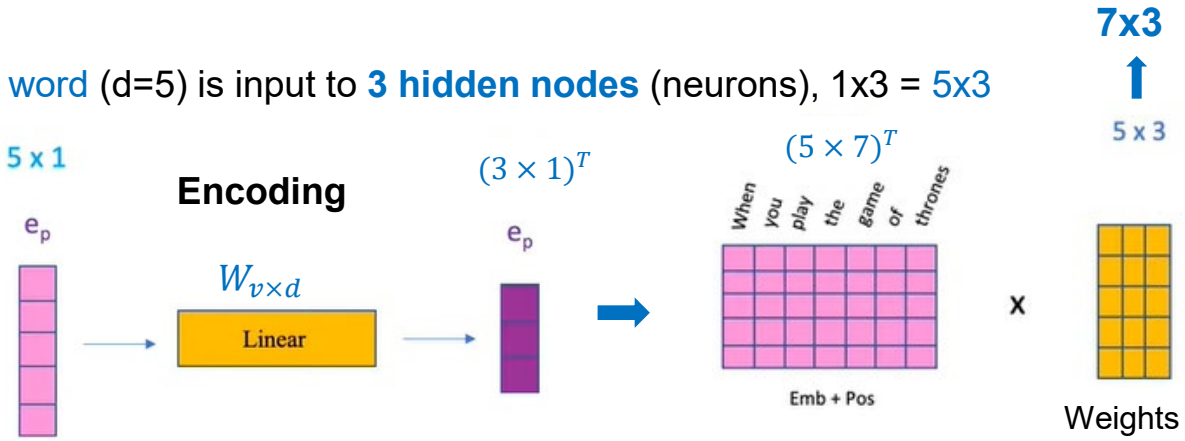
- ***Self-attention layer***
  - The **input embedding X** (with positional information) is fed to each of three **linear layers** and turned to Query (**Q**), Key (**K**), and Value (**V**).
  - To extract **context information** from arbitrarily large input without recurrent connections

What is the *Linear Layer* for?

- **The input is *projected* into *different feature spaces* Q, K, and V,**
- **to learn** a new (encoded) representation.
  - An input vector $x \in \mathbb{R}^d$ into a new vector $\boldsymbol{xW} + \boldsymbol{b}$ in $\mathbb{R}^{d'}$ (compression / expansion)
  - Commonly, to reduce the original input to a smaller vector but still capturing the contextual information
  - The neuron has no activation function, is only for transformation.

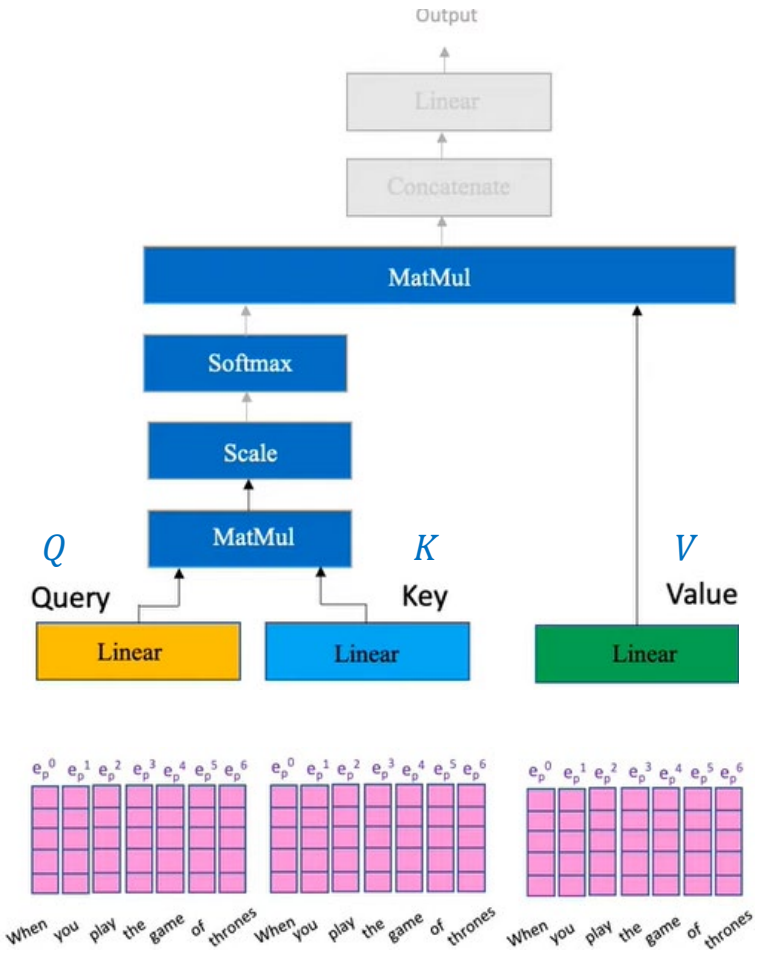


If one word (d=5) is input to **3 hidden nodes** (neurons), 1x3 = 5x3



The weight matrices at **three linear layers** $\boldsymbol{W^Q}$, $\boldsymbol{W^K}$, $\boldsymbol{W^V}$ will be used to *project* the **input embedding** into an *embedding representation* of **Q**, **K**, and **V**.

Query (Q), Key (K), and Value (V)

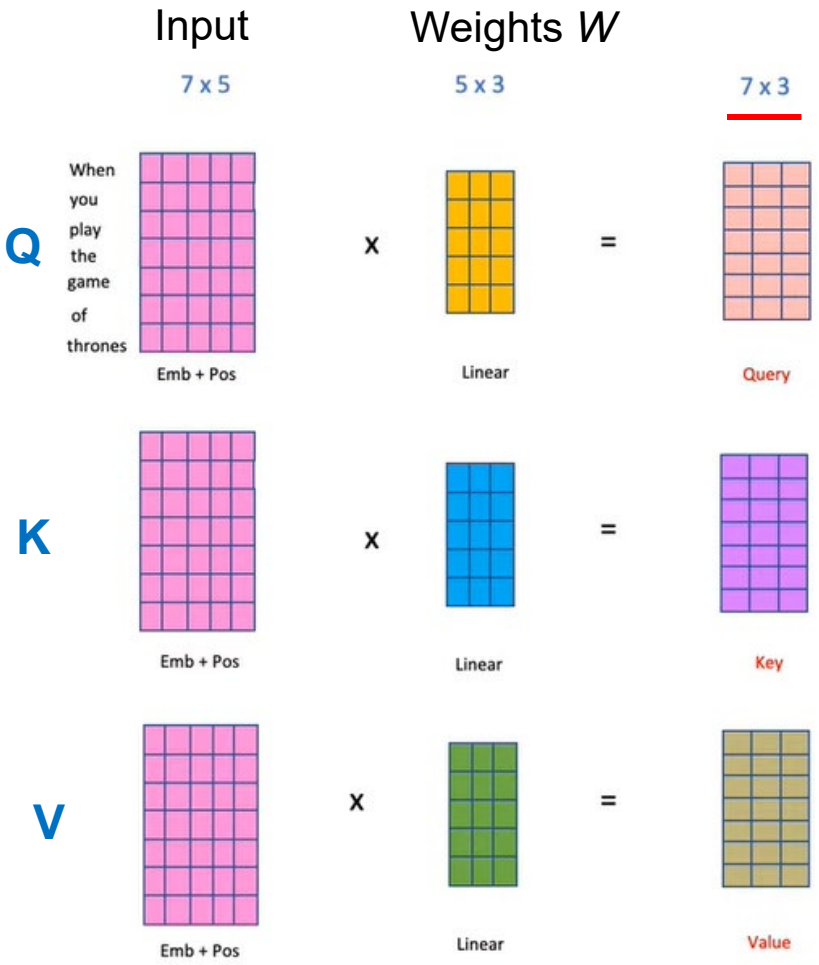


The same input sequence is fed.

- **Intuition of Q, K, and V**
  - **Q** is the current focus of attention (like a search query).
  - **K** is a preceding input being compared to the current focus (like the matching key).
  - **V** is to compute the output for the current focus (like the matched content).
- **Why matrix multiplication?**
  - To compute cosine similarity

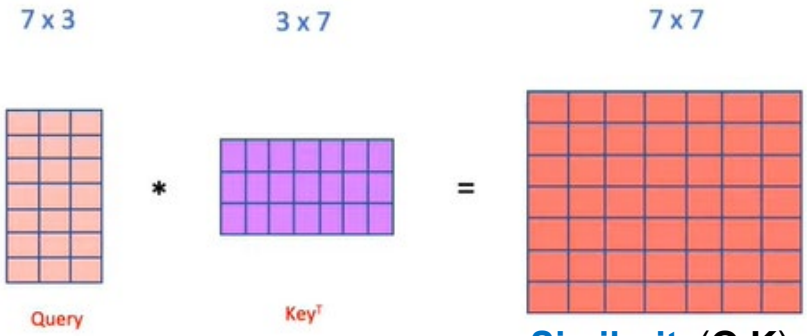
    $\cos(\theta) = \frac{A \cdot B}{||A||||B||}$ (range [-1, +1])
  - $similarity(\boldsymbol{Q}, \boldsymbol{K}) = \frac{Q \cdot K^T}{\boldsymbol{scaling}}$ why scaling?
- **Attention (Q, K, V)** = $\boldsymbol{softmax}(\frac{\boldsymbol{Q} \cdot \boldsymbol{K^T}}{\sqrt{\boldsymbol{d_k}}}) \cdot \boldsymbol{V}$
  - captures the context information for each word pair (self-attention).
    - d_k is the vector dimension.
    - Softmax calculates the attention weight assigned to tokens in V.

Query, Key, and Value Representation



- **Attention score**
  - Cosine similarity as the attention score
  - $similarity(Q, K) = \frac{Q \cdot K^T}{scaling}$
- **Attention output**
  - $Attention(Q, K, V) = Softmax(\frac{Q \cdot K^T}{\sqrt{d_k}}) \cdot V$
  - $Softmax(\frac{Q \cdot K^T}{\sqrt{d_k}})$ is attention weight that captures the context information for each word pair (self-attention).

Computing the Attention Weight



Attention score ($Q \cdot K^T$) is to filter only the important information of input sequence.

Similarity(**Q**,**K**)

Attention Filter

$$softmax(\frac{Q \cdot K^T}{\sqrt{d_k}})$$

7 x 7

	When	you	play	the	game	of	thrones
When	89	20	41	10	55	10	59
you	20	90	81	22	70	15	72
play	41	81	95	10	90	30	92
the	10	22	10	92	88	40	89
game	55	70	90	88	98	44	87
of	10	15	30	40	44	85	59
thrones	59	72	92	90	95	59	99

Softmax

7 x 7

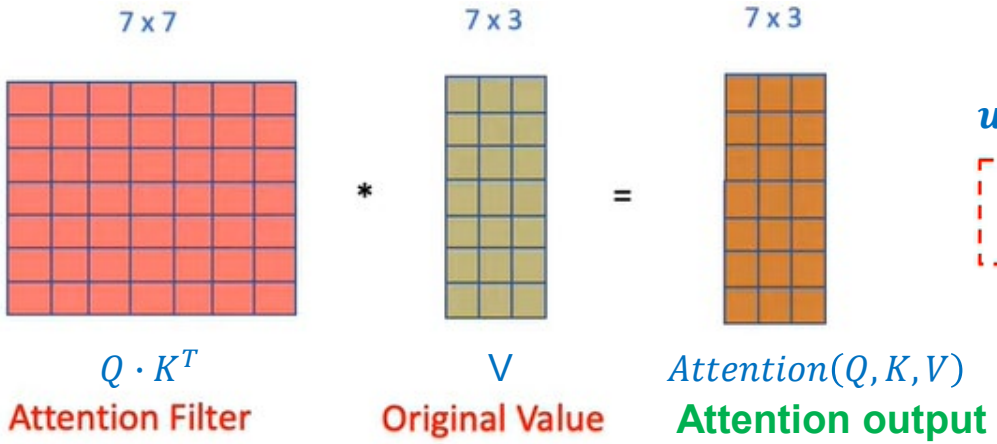
	When	you	play	the	game	of	thrones
When	33.6	7.6	15.5	3.8	20.8	3.8	22.3
you	7.6	34.0	30.6	8.3	26.5	5.7	27.2
play	15.5	30.6	35.9	3.8	34.0	11.3	34.8
the	3.8	8.3	3.8	34.8	33.3	15.1	33.6
game	20.8	26.5	34.0	33.3	37.0	16.6	35.9
of	3.8	5.7	11.3	15.1	16.6	32.1	22.3
thrones	22.3	27.2	34.8	34.0	35.9	22.3	37.4

Scaling by d_k
(the dimension of key vector)

$\sqrt{7}$

$$\text{softmax}(x_i) = \frac{\exp(x_i)}{\sum_j \exp(x_i)}$$

Computing the Self-Attention Output

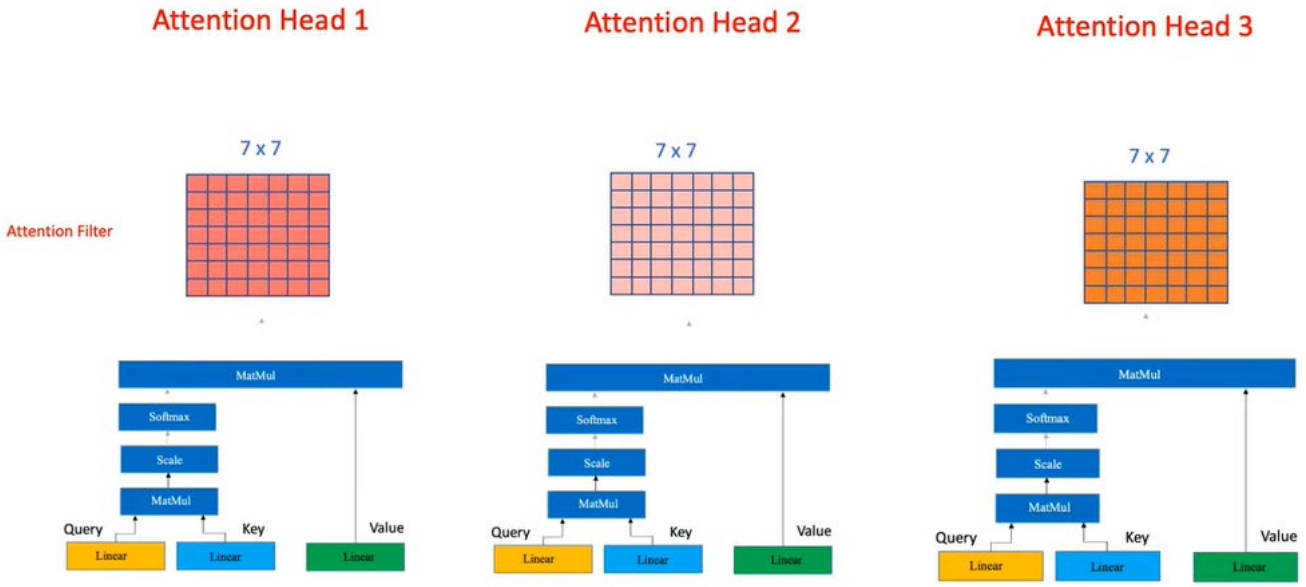


using Self Attentoin score

$$\text{Attention(Q, K, V)} = \text{softmax}\left(\frac{Q.K^T}{\sqrt{d_k}}\right).V$$

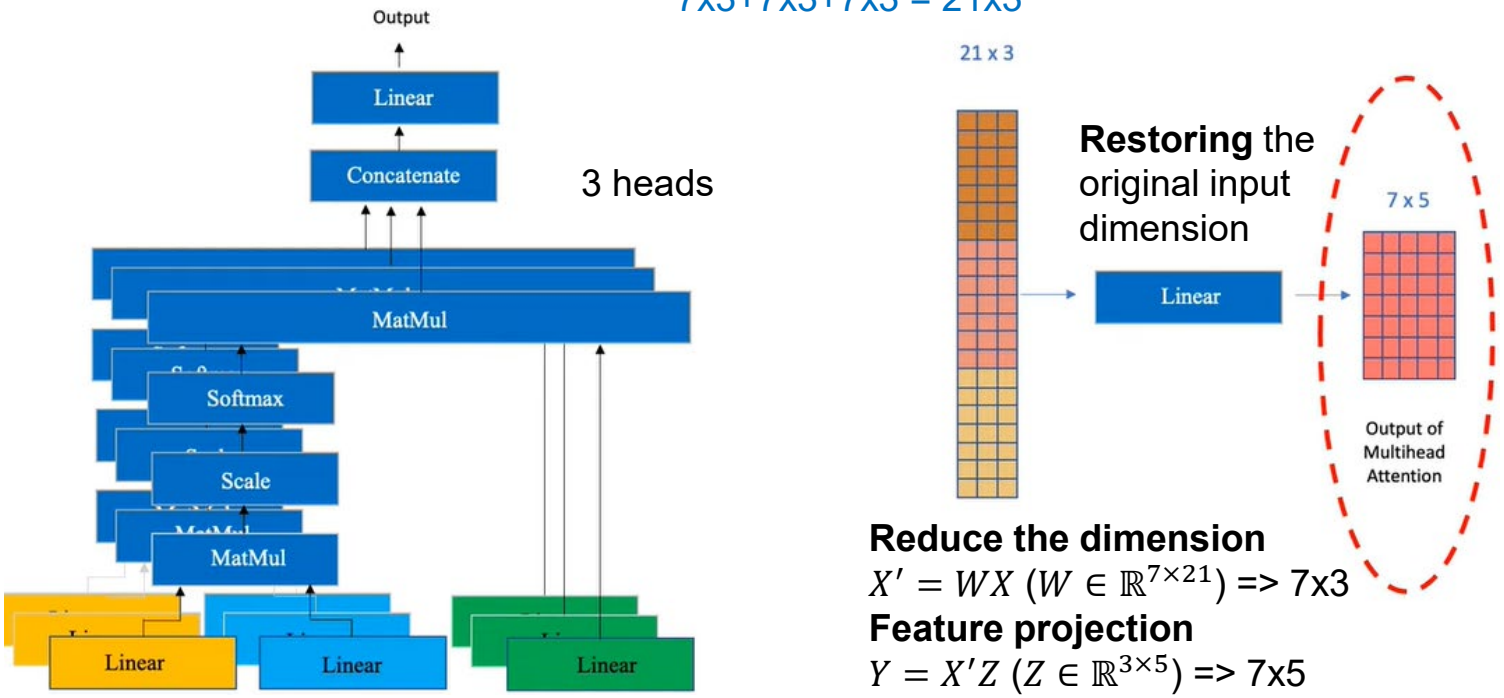
- **The encoded representation of the input sequence**
  - The filtered value by the attention score is the attention output that represents the important information (semantic, contextual relationship) within an input sequence.
  - This way, the encoded representation of the input sequence keeps only the key features without the entire input data.
  - Will $Attention(Q, K, V)$ give the largest score on itself?

Why *Multi-Head* Attention?



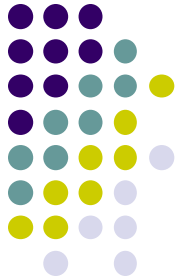
- **To capture *various features* from the input sequence**
  - Example heads
    - Semantic relationships, positional relationships, syntactic structure, etc.
    - Like multiple kernels in CNN, capturing multiple small patterns via hierarchical learning
    - Is it possible for multiple heads to learn the same aspect?
  - Parallel computation (with GPUs) can increase the model's efficiency.
    - The original transformer used 8 attention heads.
    - The larger model may use more. GPT-3 used 96 attention heads.

Output of Multi-Head Attention Layer



- **Multi-head attention layer**
  - Concatenates all those attention heads together and go through linear layer again to produce the final output.
    - Restore the original input dimension: why?
    - Reduce the dimension, followed by feature projection
    - Flatten to 63 and apply a linear transformation to get 35, and reshape 35 to 7x5

Parameters to Learn at Self-Attention



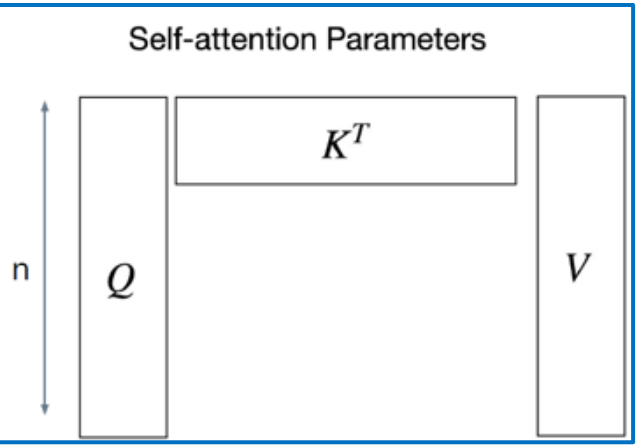
Parameters (learned): W_Q, W_K, W_V

Compute:

$$Q = XW_Q$$

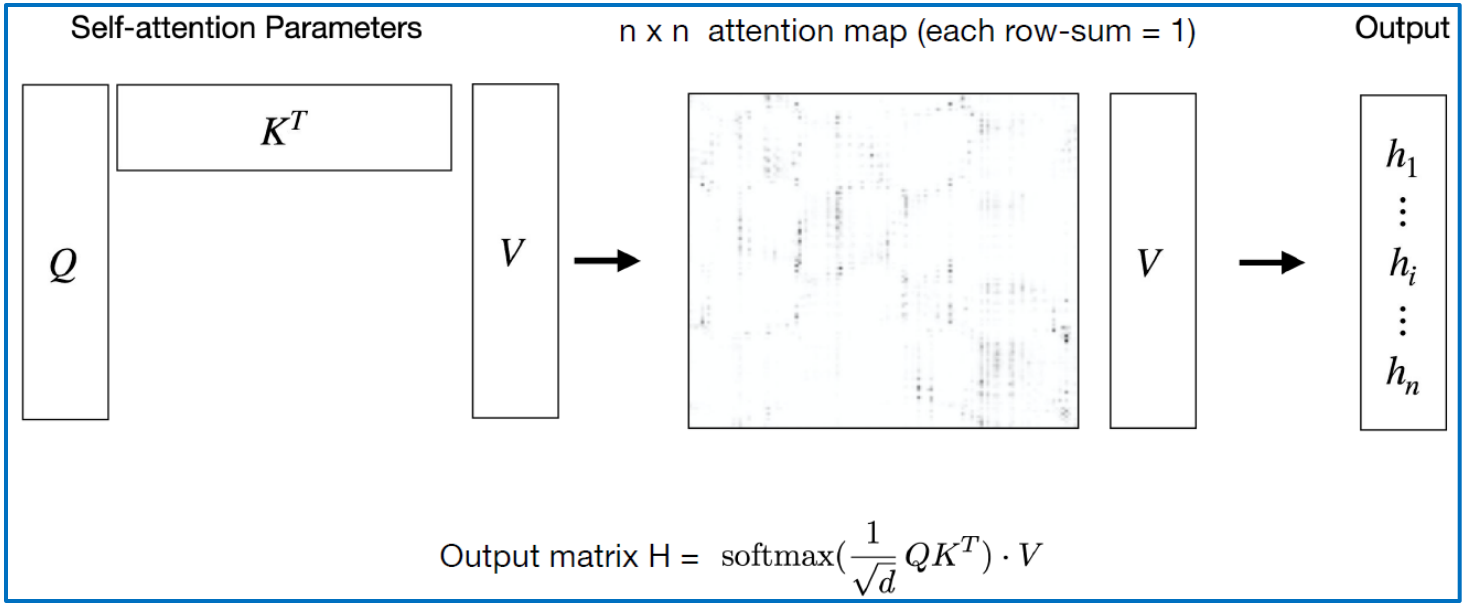
$$K = XW_K$$

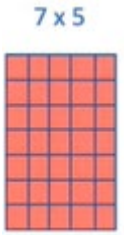
$$V = XW_V$$



- X **is input matrix**.
  - Multiple d-dimensional word embedding vectors
- W_Q, W_K, W_V are weight matrices for each **Q**, **K**, and **V**.

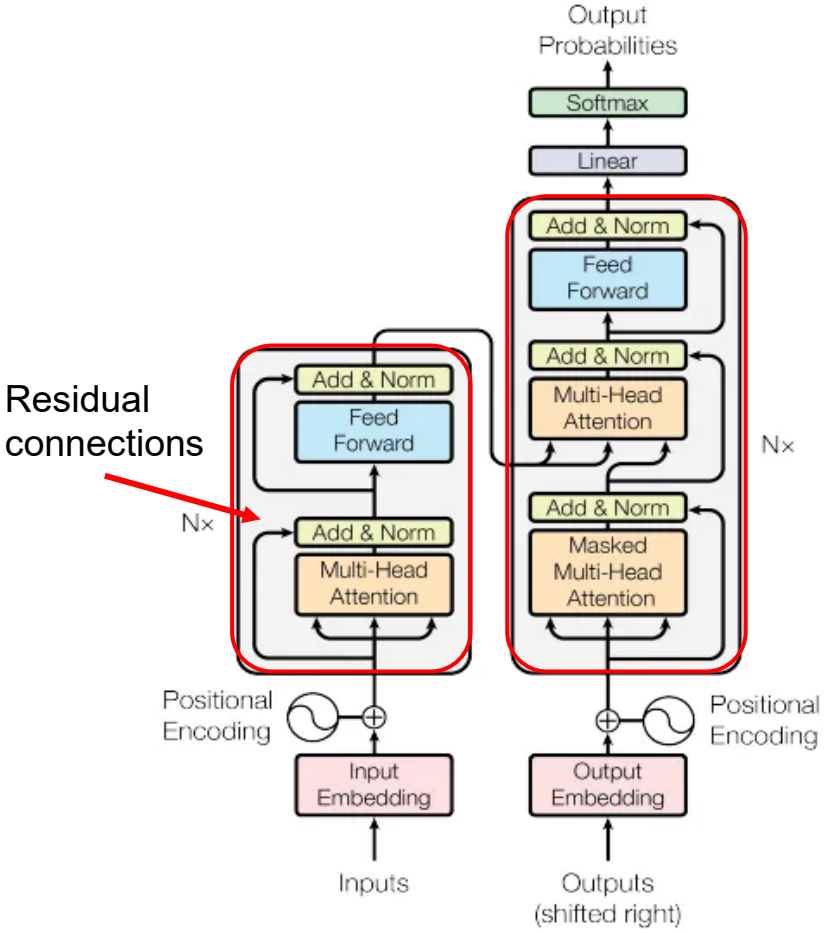
The Output of Self-Attention Layer





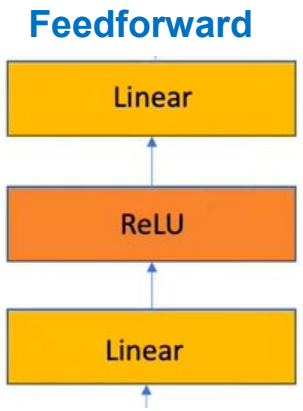
- **The output of self-attention layer** is a matrix that **contains**:
  - Contextualized token embeddings
  - Information from all tokens and learned relationships - Multi-head attention

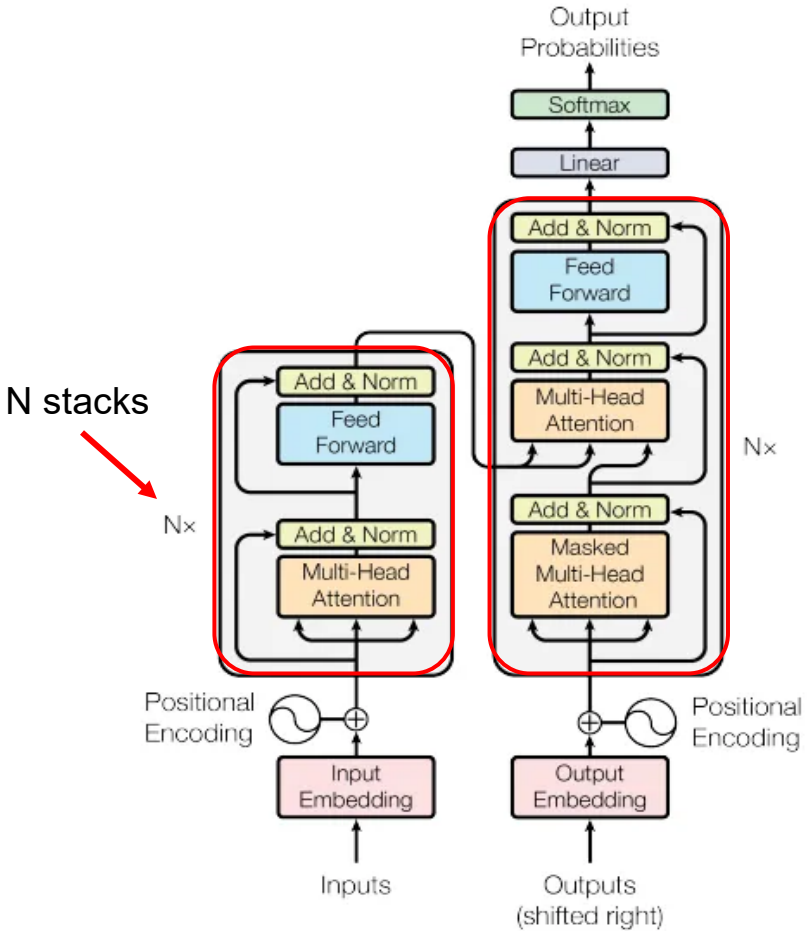
Add & Norm after Multi-Head Attention



- **Add** (**residual connection**)
  - The input **+** the output of multi-head attention
- **Norm** (**normalization**)
  - To stabilize training by reducing covariance shift
  - **Layer normalization**
    - The embedding vector of each token is standardized by mean and stdev.
    - Normalizing the activations for each sample in a batch across all the features (not across the batch).
    - Useful when the batch size is small or inconsistent (seq2seq modeling)

Feedforward Layer and N Stacks





- **Feedforward**
  - Fully-connected feedforward layer is a part of the residual connection architecture.
  - This layer is for the model to learn more complex patterns and relationships by increasing the depth (theoretically enhancing its learning capacity).
  - Feedforward is applied to each position separately and identically to maintain positional information.
- **N stacks of encoders (Nx)**
  - Stacking multiple encoders can be stacked on N times.

Transformer: Language Modeling

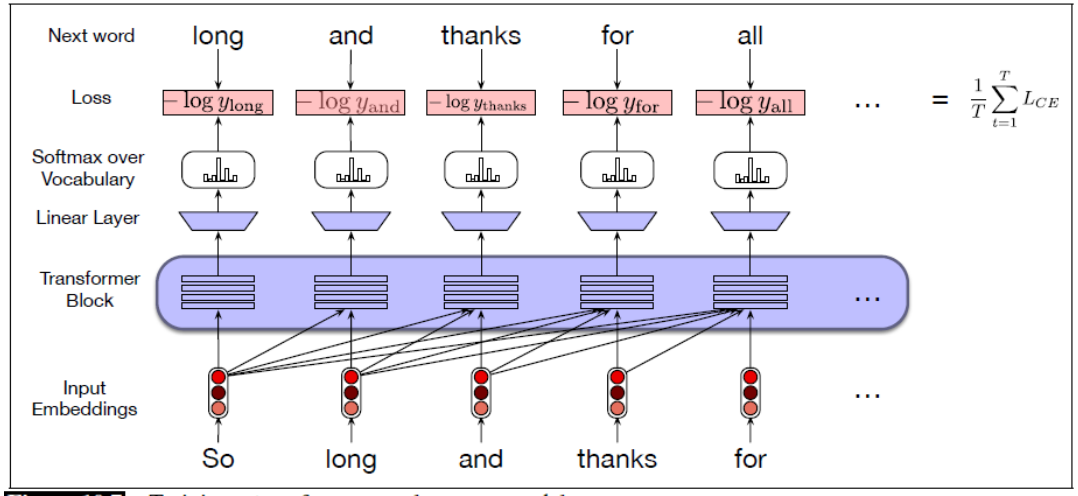
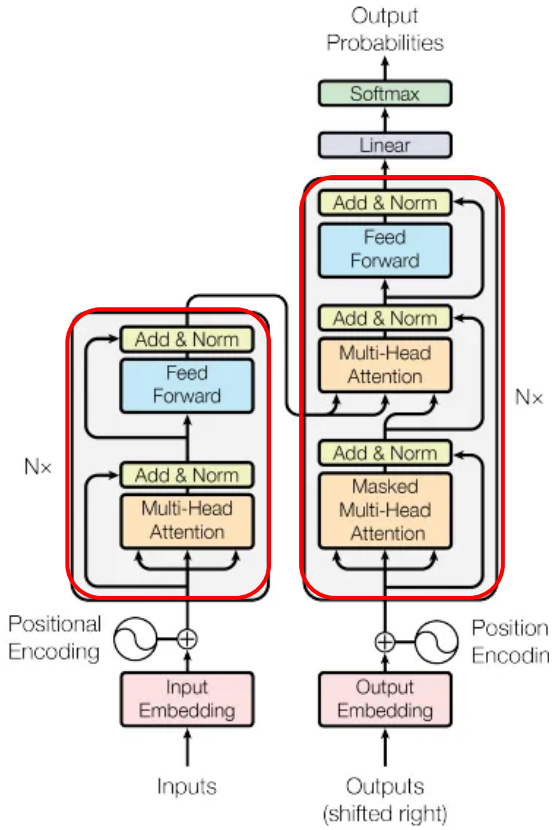


Figure 10.7 Training a transformer as a language model.

- **The basic idea of text generation by prediction with a decoder**
  1. Sample a word for the output starting from the first symbol marker **<start>**
  2. **Decoder** uses the word embedding as the input and samples (predicts) the next word in parallel with the causal mask (unlike RNN).
     - **Causal mask**: Each position only attends to itself and earlier tokens.
     - But at inference time, predict one token at a time **autoregressively**. Why not in parallel?
  3. Continue generation until the end of sentence marker **<end>**.
- **Training to predict next words**
  - $L_{CE}(\hat{y}_t, y_t) = -\log \hat{y}_t[w_{t+1}]$ using *teacher forcing* at each time step in decoding
    - Calculate the loss in *parallel across all tokens* in the sequence.
    - Backpropagate the aggregated loss over the entire sequence.

Transformer: Seq2seq Modeling



- ***Decoding* process to generate tokens**
  - **Prepare** the entire target sequence (ground truth) with <start> and <end> token added
  - Feed the sequence to the decoder as the input and shift by one token to predict the next token.
    - Enabling self-attention and parallel computation
  - **Generate** one token at a time, using the **encoder's** output (cross-attention) and feed the tokens generated so far back to the **decoder** (autoregressive).
    - The decoder produces a probability vector to chooses the token with the highest probability at that position in the sequence.
    - This selected token is then used as a part of the input to the decoder for generating the next token.
  - Repeat this autoregressive process until <end> token is generated.

Cross-Attention Layer

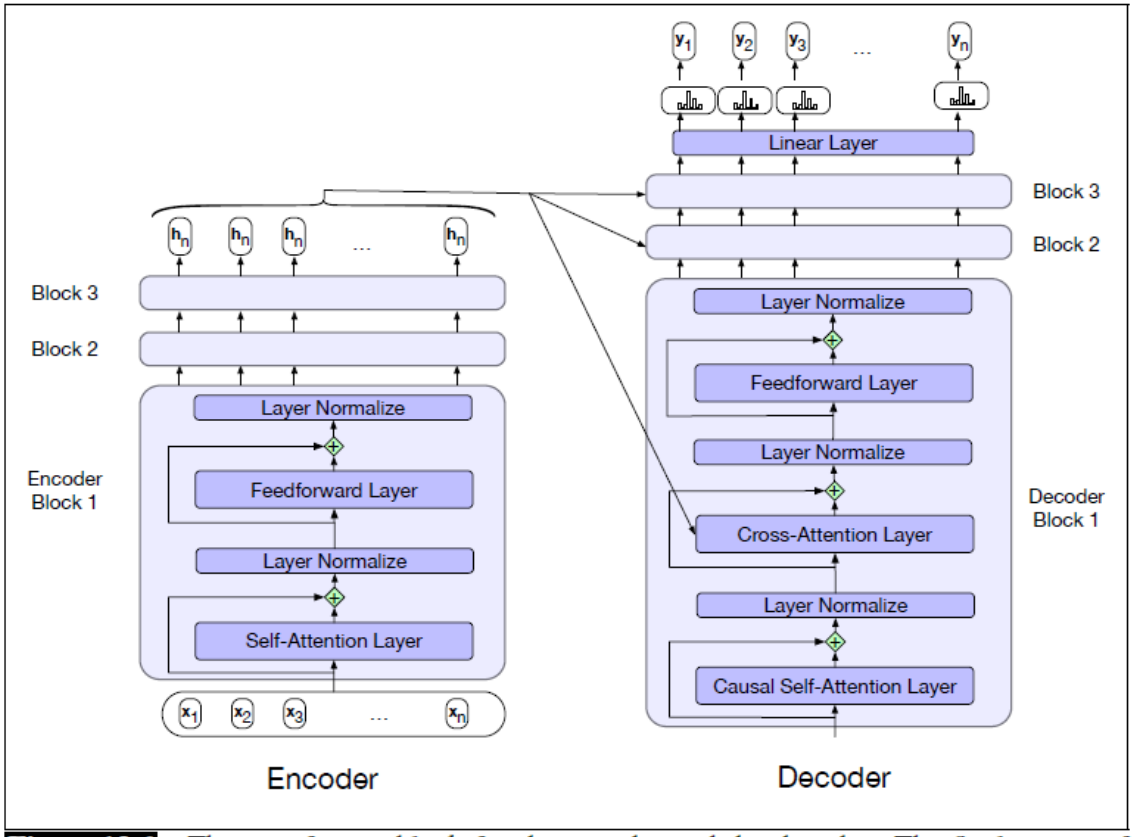
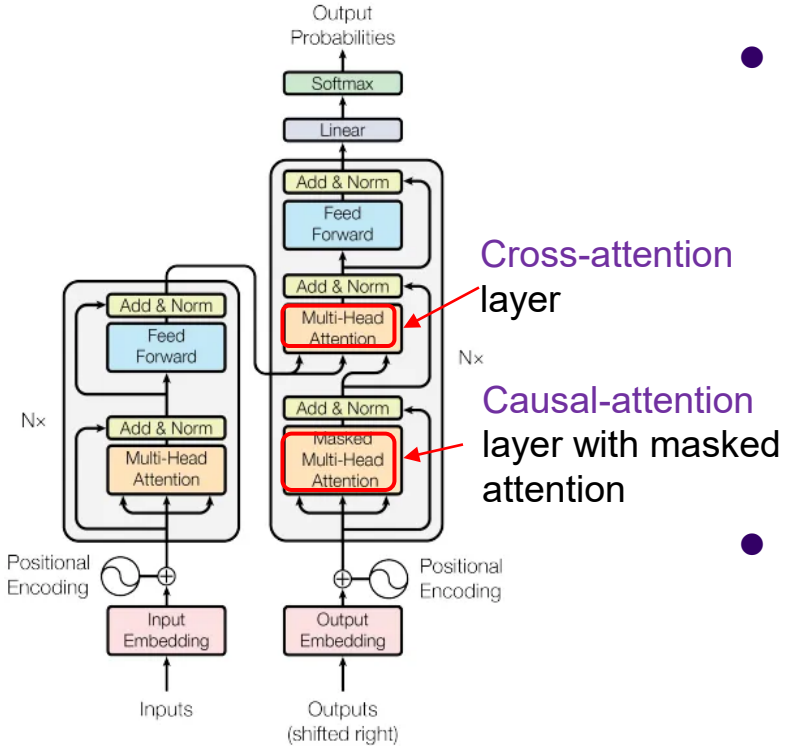


Figure 13.6 The transformer block for the encoder and the decoder. The final output of the encoder $\mathbf{H}^{enc} = \mathbf{h}_1, ..., \mathbf{h}_T$ is the context used in the decoder. The decoder is a standard transformer except with one extra layer, the **cross-attention** layer, which takes that decoder output $\mathbf{H}^{enc}$ and uses it to form its **K** and **V** inputs.

Causal-Attention and Cross-Attention



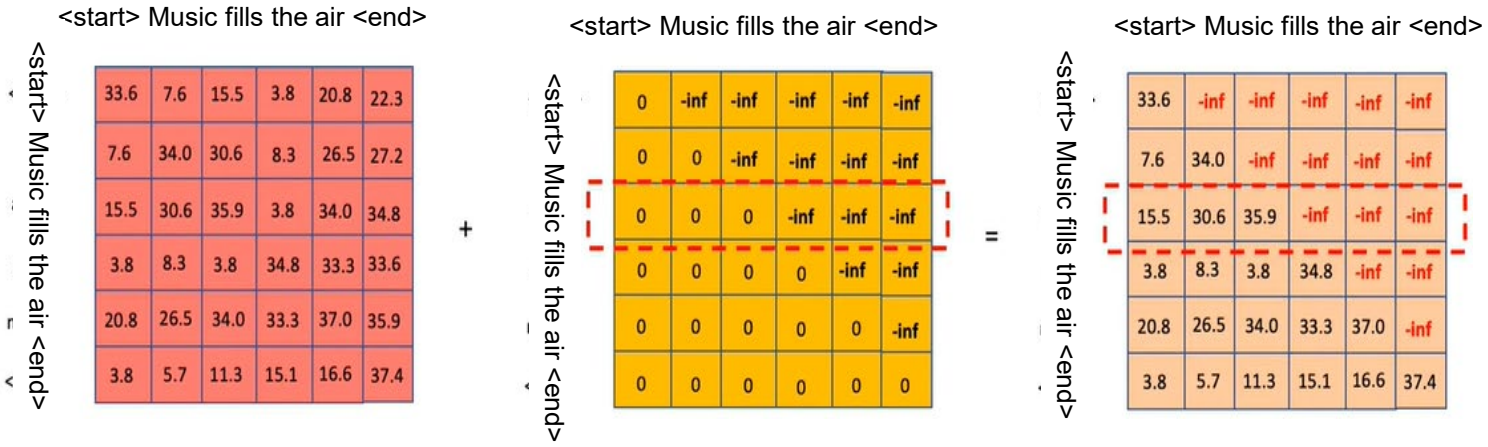
- **Causal-attention layer**
  - The decoder's self-attention attends to itself and all previous tokens focusing on the already generated output, but never future tokens.
  - Why causal attention?
    - (cause → effect) because it enforces a causal (time-ordered) dependency (causal mask).
- **Cross-attention layer**
  - The decoder attends to encoder outputs, combining them with its own embeddings to focus on relevant parts of the input sequence while generating each token coherently.
- **Still multi-heads in decoder!**

Masked Multi-Head Attention

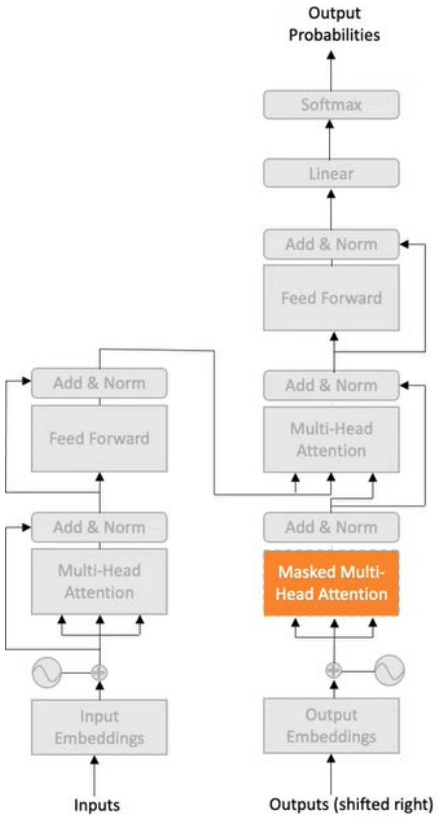
- Computing attention score requires matrix multiplication.
  - $Attention(Q, K, V) = Softmax(\frac{Q \cdot K^T}{\sqrt{d_k}}) \cdot V$
  - How can we prevent accessing future tokens (upper triangle) when the entire output token is fed to decoder?
- **Masked multi-head attention layer**
  - The values of the top right triangle are filled with 0s or negative infinities.

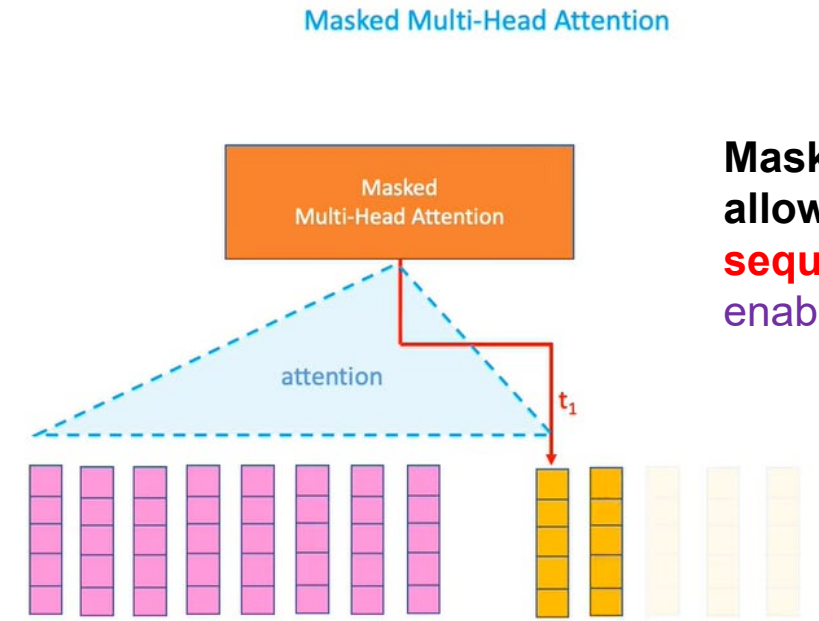
Source: <start> **When you play the guitar** <end>
Target: <start> **Music fills the air** <end>

Masked-attention filter



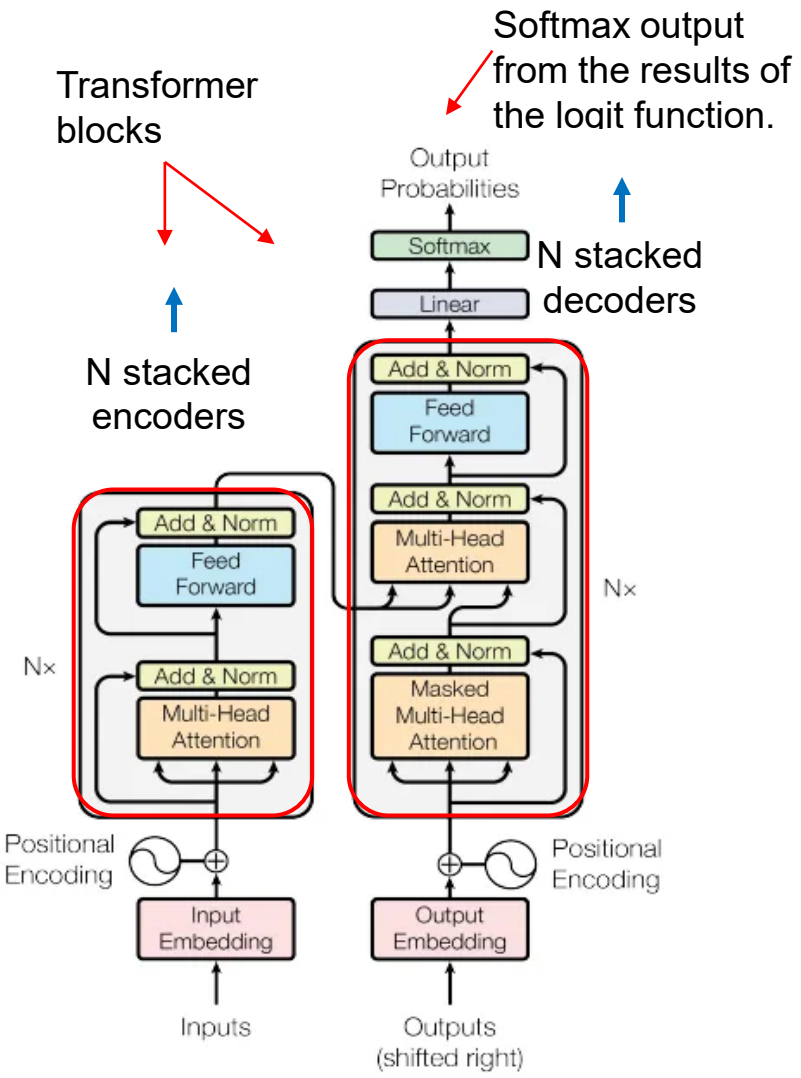
Masked Multi-Head Attention





Masked multi-head attention allows feeding the **entire sequence** to the decoder, enabling parallel processing.

The *Output* of the Transformer



- **The output of final layer**
  - A vector of probabilities that allows choosing the most likely token
- **Transformer blocks**
  - A stacks of transformer blocks where each block is a transformer network.

Transformer Blocks

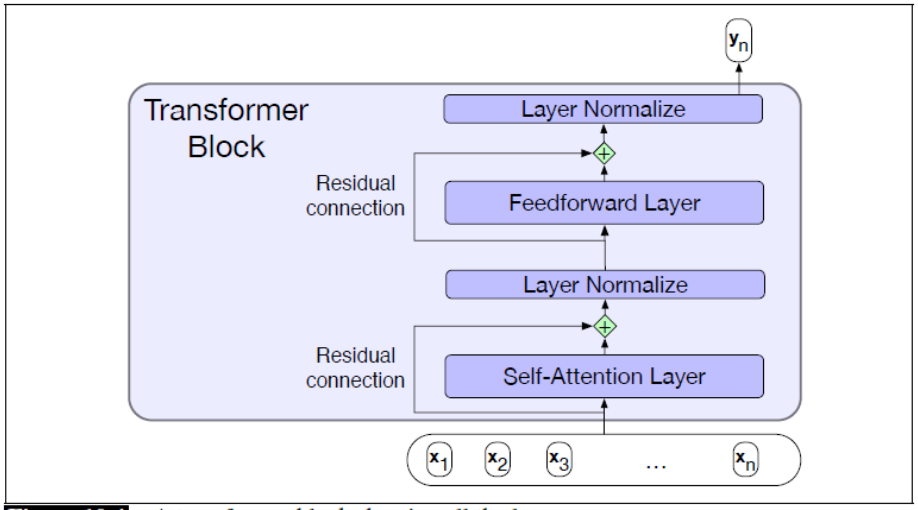


Figure 10.4 A transformer block showing all the layers.

- **Variations of transformer**
  - Transformers with only encoder portion or decoder portion, a stack of encoders or decoders, or both.
  - Encoder only
    - Encoders only can do text classification, NER, feature extraction, sentence embeddings, etc.
  - Decoder only
    - Decoders only can do text generation, language modeling, etc.

- **Transformers blocks**
  - Stacks of transformer blocks
  - Each block is a multilayer network made by combining simple linear layers, feedforward networks, and self-attention layers.
- **Easier parallelization**
  - Computation performed for each item is independent of all the other computations.

Bidirectional Transformers Encoders

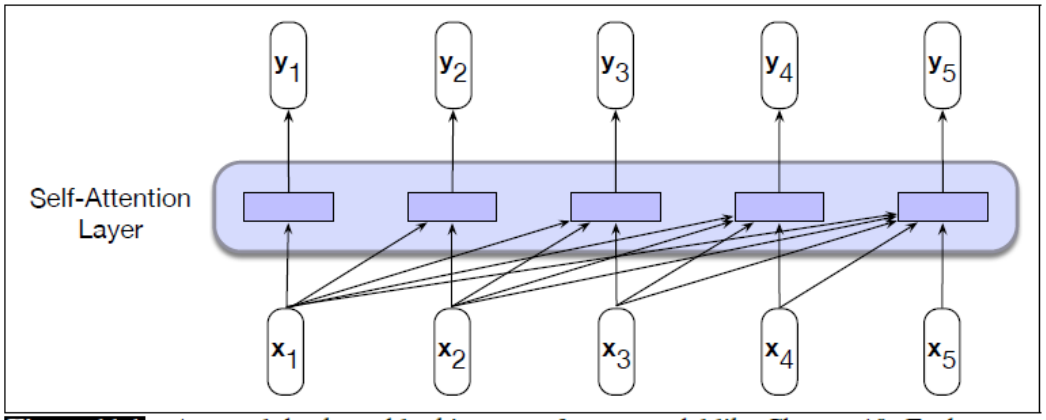


Figure 11.1 A causal, backward looking, transformer model like Chapter 10. Each output is computed independently of the others using only information seen earlier in the context.

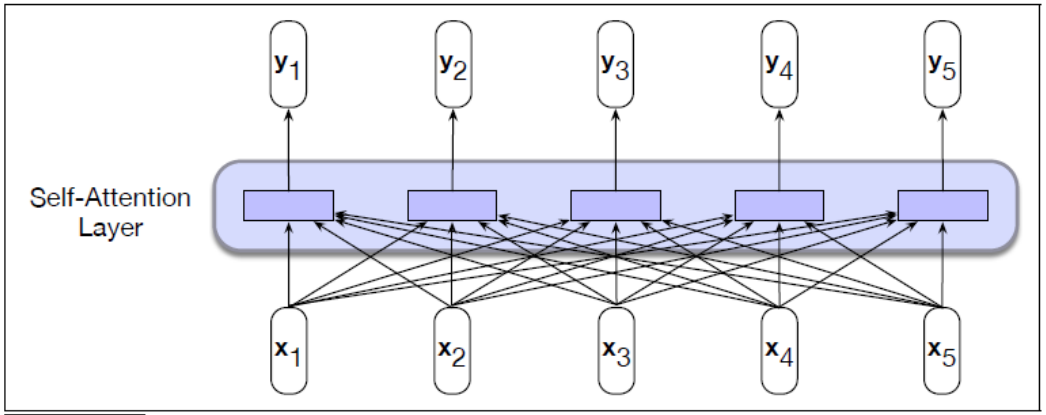


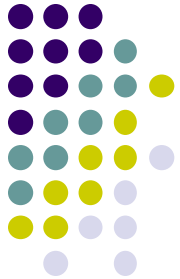
Figure 11.2 Information flow in a bidirectional self-attention model. In processing each element of the sequence, the model attends to all inputs, both before and after the current one.

BERT (Bidirectional Encoder Representations from Transformers (Jacob et al., 2018)

Descendants of BERT:
RoBERTa (Liu et al., 2019)
SpanBERT (Joshi et al., 2020)

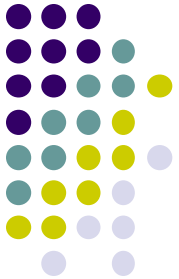
Many other variations
GPT (Open AI)
Gemini (Google)
LLaMa (Meta)
Etc.

Encoder Only Transformer



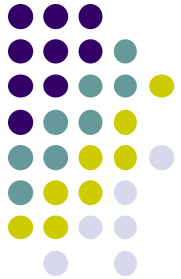
- **Encoder-only architecture**
  - Understanding (discriminative)
- **Major NLP tasks**
  - Text classification
    - Use [CLS] token embedding as classification head
  - Named Entity Recognition (NER)
    - Token-level classification using contextual embeddings
  - Part-Of-Speech Tagging
    - Token-wise predictions from encoder states
  - Question Answering (Extractive)
    - Encoder finds span boundaries in context
  - Text entailment/Natural Language Inference (NLI)
    - Checking if the truth of the premise guarantees the truth of the hypothesis (entailment)
    - Concatenate premise + hypothesis; classify relation
  - Sentence similarity/semantic search
    - Encode sentences, compare embeddings (cosine similarity)

Decoder Only Transformer



- **Decoder-only architecture**
  - Generation (generative, autoregressive)
- **Major NLP tasks**
  - Autoregressive Language Model (LM)/text completion
    - Predict next token given previous ones
  - Story/code generation
    - Free-form or structured text generation
  - Abstract QA (generative)
    - Generate textual answers (human-like answer that requires reasoning, suitable for open-domain QA, instead of finding a matching span)
  - Dialogue generation/chatbots
    - Open-ended generation
  - Multimodal chat/vision LM
    - LLM decoder takes embeddings from the vision encoder.
    - Decoder only with adapters

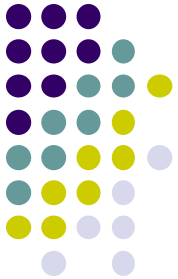
Encoder-Decoder Transformer



- **Encoder-Decoder architecture**
  - Transformation (seq2seq)

- **Major NLP tasks**
  - Paraphrase generation
    - Input → rephrased output (restating the same meaning using different words)
  - Text simplification/style transfer
    - Controlled text rewriting (rewriting text under strict constraints and controls, e.g., formality, sentiment, reading-level, domain, etc.)
  - Dialogue generation or chatbots
    - Open-ended generation
  - Abstract QA (generative)
    - Generate free-form textual answers
  - Automatic speech recognition
    - Acoustic encoder (acoustic features) → decoder (text)
  - Image captioning (cross-modal)
    - Vision encoder (image features) → decoder (text)

Transformer-based Language Models



- **What can transformers do?**
  - The *fundamental ideas* of NLP and *mathematical ingenuity* => **huge success!**
  - Most NLP tasks
  - Everything RNN or LSTM can do but better
    - Recommendation systems, video/image classification (using only encoder, image patching, convolution layer, e.g., Vision Transformer), QA systems, reasoning, etc.
- **Many Large Language Models (LLMs)**
  - BERT (Bidirectional Encoder Representation from Transformers) by Google
  - GPT (Generative Pre-trained Transformer) by OpenAI
  - Gemini (formerly known as Bard) by Google
  - Llama (Large Language Model) by Meta AI
  - Many more
- **Some issues**
  - Computationally extremely expensive for long documents
    - Attention is quadratic in the length of the input.
  - Finding more efficient algorithms is an ongoing research problem!